

Media Engineering and Technology Faculty
German University in Cairo



3-D Simulation for a Dexterous Robot Arm

Bachelor Thesis

Final Draft

Author: Omar Mahmoud Ramadan Mansour

Supervisors: Dr. Alaa Mohamed Khamis

Submission Date: 22 May, 2012

Media Engineering and Technology Faculty
German University in Cairo



3-D Simulation for a Dexterous Robot Arm

Bachelor Thesis

Final Draft

Author: Omar Mahmoud Ramadan Mansour
Supervisors: Dr. Alaa Mohamed Khamis
Submission Date: 22 May, 2012

This is to certify that:

- (i) the thesis comprises only my original work toward the Bachelor Degree
- (ii) due acknowledgement has been made in the text to all other material used

Omar Mahmoud Ramadan Mansour
22 May, 2012

Contents

1	Introduction	7
1.1	Motivation	7
1.2	Objectives	8
1.3	Thesis structure	8
2	Industrial Robotics	9
2.1	Introduction	9
2.2	Applications	9
2.2.1	Material Transfer	9
2.2.2	Welding	10
2.2.3	Spray Coating	12
2.2.4	Product Assembly	12
2.3	Classifications	13
2.3.1	Cartesian Robot	13
2.3.2	Cylindrical Robot	14
2.3.3	Spherical or Polar Robot	16
2.3.4	Articulated Robot	16
2.4	Kinematics	18
2.4.1	Overview	18
2.4.2	Kinematic Chains	18
2.4.3	The Denavit-Hartenberg Convention	20
2.4.4	Forward Kinematics	20
2.4.5	Inverse Kinematics	21
2.5	Challenging problems	22
2.5.1	Environment perception	22
2.5.2	Path planning	23
2.5.3	Object grasping	24
3	Simulation Environment for Industrial Robots: A Review	25
3.1	Introduction	25
3.2	Simulation Environment Requirements	26

3.3	Survey on Simulation Environment	27
3.3.1	Gazebo	28
3.3.2	Webots	28
3.3.3	Modular Open Robots Simulation Engine	30
3.3.4	Simbad	32
3.3.5	Microsoft Robotics Developer Studio	33
3.4	Comparative Study	36
4	Proposed System	37
4.1	Introduction	37
4.2	System Block Diagram	37
4.2.1	Open Robotics Automation Virtual Environment . . .	37
4.2.2	Python Programming Language	40
5	Experimental Study	47
5.1	Experimental Setup	47
5.2	Evaluation Metrics	47
5.2.1	Quantitative Evaluation	47
5.2.2	Qualitative Evaluation	54
5.3	Results and Discussion	62
6	Conclusion and Future Work	63
6.1	Conclusion	63
6.2	Future Work	63

List of Figures

2.1	Example of typical pick and place robot arm.	10
2.2	Standford arm invented by Victor Scheinman in 1969.	11
2.3	Spot welding process.	11
2.4	Arc welding process.	12
2.5	Cartesian Traverse Robot diagram	14
2.6	XRS System - Traverse Robot	14
2.7	Cylindrical Robot arm diagram	15
2.8	Cylindrical Robot arm	16
2.9	Spherical Robot arm diagram	17
2.10	Spherical Robot arm	17
2.11	Articulated Robot arm diagram	18
2.12	Types of Robot Kinematics	18
4.1	Proposed System Block Diagram	37
4.2	The OpenRAVE Architecture is composed of four major layers and is designed to be used in four different ways	39
4.3	The proposed system environment	40
4.4	Proposed Barrett-Wam robot arm	41
4.5	Real Barrett-Wam robot arm	41
5.1	Relative Point Position with Success Rate Line Graph	49
5.2	Relative Joint Angles with Success Rate Line Graph	51
5.3	Relative Joint Angles with Time Line Graph	52
5.4	Environment Complexity with Success Rate Line Graph	53
5.5	Environment Complexity with Time Line Graph	54
5.6	General Control Panel	55
5.7	Barrett-Wam in minimum complexity environment	56
5.8	Barrett-Wam in medium complexity environment	56
5.9	Barrett-Wam in medium complexity environment	56
5.10	Barrett-Wam in maximum complexity environment	56
5.11	Direct Control panel	57

5.12	Example shows Direct Control - Opened Fingers	58
5.13	Example shows Direct Control - Closed Fingers	58
5.14	Example shows Direct Control-Object Grasping	58
5.15	Example shows Direct Control-Object Grasping	58
5.16	Example shows Direct Control with joint 2 with angle equals 82 units	59
5.17	Forward Kinematics panel	60
5.18	Forward Kinematics Example 1	60
5.19	Forward Kinematics Example 2	60
5.20	Forward Kinematics - Unreachable destination Example	61
5.21	Forward Kinematics Example - Top View Camera	61
5.22	Inverse Kinematics Panel	62
5.23	Inverse Kinematics Example	62

List of Tables

Chapter 1

Introduction

1.1 Motivation

Robotics is the technology associated with the design, construction and operation of robots and automated machines that can move and react to sensor input. It is also considered as the technology of using robots to solve problems. It is one branch of artificial intelligence. Robots are used in a wide range of industrial applications. The basic concept of industrial robots is that they are re-programmable, manipulatable in three or more axes. An industrial robot has at least three degrees of freedom (3-DOF). Material handling, spot welding and spray painting were the earliest applications concerning industrial robots. Nowadays pick and place, assembly, product inspection and testing have been added to the industrial applications. Endurance, speed and precision are the three principle measurement for a robot application. Modern industrial robots are true marvels of engineering. A robot of a size of a normal person can easily carry a load over one hundred pounds and move it very quickly with a repeatability of ± 0.006 inches. Furthermore these robots can do that 24 hours a day for years on end with no failures. Though they are re programmable however in many applications they are programmed once and then repeat that exact same task for years.

The basic idea of robot simulation is having an animated robot running within an environment without even caring about the real physical robot. So generally robot simulator is an application that presents to the user a virtual robot with a virtual environment and that is capable of emulating the motion generation of the robot and how this robot can interact with this environment. The use of a robotics simulator for development of robotics control program is highly recommended regardless of whether an actual robot is

available or not. Furthermore the simulation environment provides an ideal testing bed for a new control algorithms. More importantly the simulation provides a very user-friendly teaching environment for robot behavior and control study. The thesis aims to cover and review all the available applications that can be used in robot simulation.

1.2 Objectives

The primary objective of the thesis is the development of a full robot simulation system which includes modeling an industrial robot, simulation for the environment and controlling the robot itself, allowing the user to interact with the robot and the environment through a user interface application. It also checks how efficient was the resulting interaction from the control and how it could be improved in future works.

1.3 Thesis structure

The thesis contains mainly 6 chapters. Chapter 1 is an introduction about the robotics simulation system, Chapter 2 presents a brief overview and description about the industrial robotics addressing the different geometric models available, their applications and classification, forward and inverse kinematics of a manipulator, and finally the basic challenging problems which include environment perception, path planning and object grasping.

Chapter 3 is proposing a review on the simulation environment for industrial robots and their requirements. Also a survey on simulation environments will be included and discussed. The proposed system and the hierarchy of the system will be discussed in chapter 4. A detailed explanation about each and every component in subsections will be included.

Chapter 5 will discuss experimental study with 3 main topics: experimental setup, evaluation metrics, results and discussion. Finally the conclusion and the future work will be mentioned in chapter 6.

Chapter 2

Industrial Robotics

2.1 Introduction

The International Organization for Standardization (ISO) and the International Federation Of Robotics (IFR) have defined the industrial robot as an automatically controlled, re-programmable multipurpose manipulator programmable in three or more axes. Another definition for the industrial robot by the Robot Institute of America is a re-programmable multi-functional manipulator designed to move materials, parts, tools or specialized device through variable programmed motions for the performance of a variety of tasks. Also the Japanese Industrial Robot Association (JIRA) came up with another definition which is a device with degrees of freedom that can be controlled.

2.2 Applications

2.2.1 Material Transfer

With simple automating the industrial robots, they can easily pick a part up and place it into a new location. Speeding up the process is not the only profit that is gained from the pick and place robots, but they also increase the production rates, the product become more accurate and the robots themselves do not fatigue. Most of the movements that pick and place robots perform are back-breaking or hard to maneuver for humans. The consistent output of a robotic system along with quality and repeatability are unmatched. Pick and place robots can be re-programmable and tooling can be interchanged to provide for multiple applications.

Long term savings are associated with a pick and place robotic workcell.

Besides, factories money has been saved by an increase in output with a material handling robotic system. With the advancements in technology and afford-ability of robots, and for automation applications, more pick and place robotic cells are being installed.

In most cases, the pick and place robot has a single arm with shoulder, elbow, and wrist joints, as well as the end effector which can called also the hand. The end effector is usually a gripper in order to pick the desired object. Those robots are often run openloop using mechanical stops or limit switches to determine how far in each direction to go. An example is shown in (Figure 2.1), pneumatic cylinders are used by the robot to lift, rotate, and extend the arm. It also can be programmed in order to repeat a sequence of given operations.

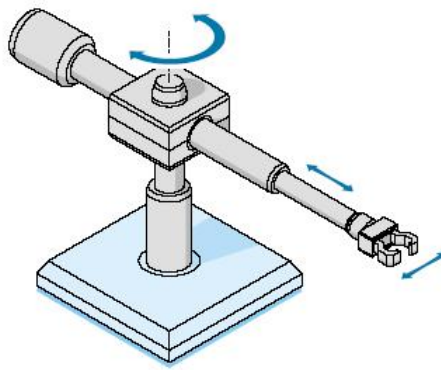


Figure 2.1: Example of typical pick and place robot arm.

2.2.2 Welding

In 1969 Victor Scheinman invented the Stanford arm (Figure 2.2), it is an articulated all-electric with 6 axis robot which is designed to permit an arm solution. Basically the Stanford arm is the main project and the main idea which has been developed into more sophisticated applications such as assembly and welding. There are two general types for welding, spot welding and arc welding (Figure 2.3 and 2.4 respectively). The idea of spot welding is to know exactly the point at which welding will be applied. A high electric current that passes through the 2 sheets in the welding spot causes a fusion.

On the other side, the idea of the arc welding is that the robot holds an electrode and moves along a specific path over the 2 sheets to be welded.

Whenever the electrode touches the metal sheets, a very high electric current passes that causes the welding. The main disadvantage of the arc welding is that it is usually harmful for humans because it produces ultraviolet rays. So that is a perfect task to be done by robots.

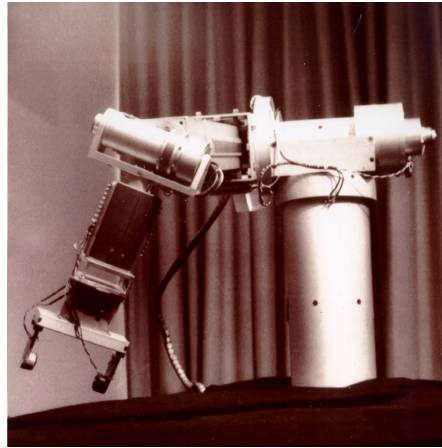


Figure 2.2: Stanford arm invented by Victor Scheinman in 1969.

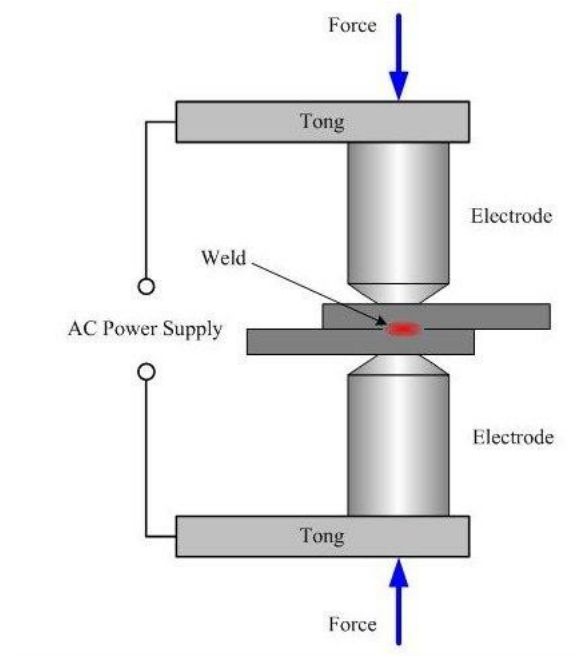


Figure 2.3: Spot welding process.

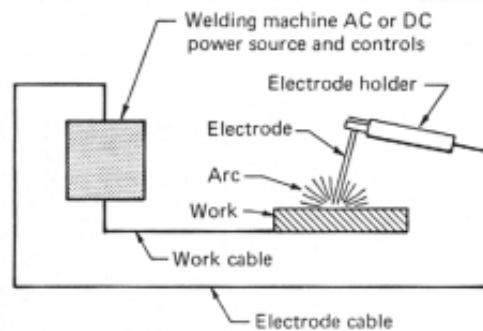


Figure 2.4: Arc welding process.

2.2.3 Spray Coating

The process or the application of spray coating is that the robots are used to change the color of a certain product. Moreover, for coating parts with resistant and protective substances, the thermal spray application is used. The coating material can be easily changed on the part. Regardless of the size and the complexity, the industrial coating robots can be programmed to easily coat all kinds of products. In a production line, time and money could be saved by the incorporating automation. Generally, Thermal Spray Robots are equipped with six axes. Three axes for the base motions and three for applicator orientation.

For guidance, machine vision could be incorporated by some thermal spray units, it can also checks application quality. Thermal spray robots today are universal, electrically driven, rather than hydraulically or pneumatically powered. As Arc welding, the spray coating is unsafe for humans due to the fumes and the mist in air. So spray coating is a typical task to be done by robots rather than by humans.

2.2.4 Product Assembly

Along an industrial assembly line, a variety of repetitive tasks have been performing by the assembly robots. A robotic arm that is fitted with an articulated hand or tool on the end is a typical assembly robot. So they are an efficient and durable alternative to human workers, as they are precise and do not experience fatigue. Manufacturing can be performed on a massive scale and with few errors when there exist many robots working at once.

Robots are also desirable to manufacturing companies as a means to reduce labor costs. Some of them are used to press two or more parts together as a product moves down the line toward completion. A more complex fit-

ting might require the robot to tighten screws, apply adhesive or weld. In addition to fitting parts, an assembly robot can be used to inspect, label and package products on the assembly line.

Not only are assembly robots used for their speed and efficiency, they are also a way to cut labor costs in a factory. They can be an expensive initial investment for a company. Over time, however, factories can save a substantial amount of money by replacing human workers with assembly robots. Although maintenance is required for the robots, they are generally tireless and large manufacturing firms can see a fast return of investment.

It is also possible that the computer softwares are used to teach assembly robots how they could move. This software can run in an external computer that controls the robot, in the robot itself or both. Some assembly robots only require a simplistic, preprogrammed sequence of motions. On the other hand, other assembly robots will have a degree of artificial intelligence, making them more adaptable to varying work conditions and tasks.

2.3 Classifications

2.3.1 Cartesian Robot

A cartesian robot, also known as a cartesian coordinate robot, is a common type of industrial robot. It has three arms that each function along linear axes of control. Each of these axes is at a right angle to the other two. Computer numerical control (CNC) machine is a common application for cartesian robot, it can also have a multitude of uses, especially as either a milling or drawing machine.

The basic form of a cartesian robot consists of three arms. Each arm can move only along a two dimensional axis, backwards or forwards or, up or down but that's if the arm is vertical. Each arm is at a right angle to the other two, though, which allows the robot to utilize the motions of all three arms to reach various points in a three-dimensional space. Depending on the purpose of the robot, these arms could vary tremendously in size. Beside, the horizontal arm will have support on both ends in some particularly large designs. This is called a typical gantry robot.

The cartesian robot has an advantage over other types of industrial robots is that, rather than rotational, all three of its axes of control are linear. It

is an advantage because it greatly simplifies the robot's arm solution. Also, the programmer must be able to program the robot to move along its axes of control to reach the various desired positions, in order to program a robot for a specific task. Determining this arm solution requires calculations to determine the desired positions relative to the robot's axes of control. Linear calculations are far easier to calculate, because the programmer can perform these calculations in a closed form using basic trigonometric principles.

As a result of the cartesian robot's ability to reach different points in a three-dimensional space with relative ease, its most common application is as a CNC machine. CNC machines use computer programs to extract the necessary commands to cause the robot to function in the desired manner and then to load these commands into the robot. They enable the robot to move very precisely and thus make cartesian robots suitable for different drawing and drafting functions. When a cartesian robot is used in this manner, a tool can be lowered onto or raised off of a surface while moving along the X and Y planes to create a specific design.

However, the cartesian robot has two main disadvantages which are accessing to work envelope by overhead crane or other material handling equipment may be impaired by the robot supporting structure and difficult maintenance.

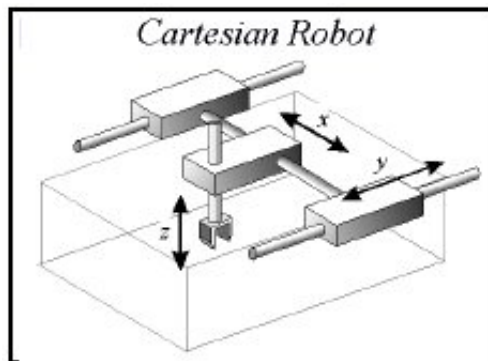


Figure 2.5: Cartesian Traverse Robot diagram



Figure 2.6: XRS System - Traverse Robot

2.3.2 Cylindrical Robot

The cylindrical robot type is one of the rarest nowadays. However, there are areas where they could still be useful. Also, they are always mentioned

when stationary robot types are discussed. Figure 2.7 and Figure 2.8 show the basic structure of this type of robots. It has three axes of movement, two of which are linear and one circular. So, usually robots of this type can move along Z and Y axes and rotate along Z axis. Basically, this forms a cylindrical coordinate system hence it has a cylindrical work envelope.

The motion of the main arm is up and down. The robot can perform this motion by extending a cylinder that is built into the arm. In most cylindrical robots, the up-and-down motion is provided by a pneumatic cylinder, and the rotation is generally provided by a motor and gears. Any part of the robot that is moved by cylinders will generally move until it hits a stop. The location of the stop is determined by placing stop blocks or location pins. The cylinders are moved by pneumatic energy (air pressure) that's controlled by simple solenoid valves. The controller determines the motion of the rotation by energizing the motor until the encoder determines the correct amount of movement has occurred.

Additional movement can be achieved by attaching a wrist to the end of the arm cylinder. In some robots the wrist is complex enough to provide one or more additional degrees of freedom. These are called pitch (up-and-down motion at the wrist), roll (rotational motion at the wrist), and yaw (side-to-side motion at the wrist). Wrists are available with one, two, or three of these motions depending on the expense of the robot and the application it's used for. However the cylindrical robot arm has a disadvantage which is limited reachability to left and right side.

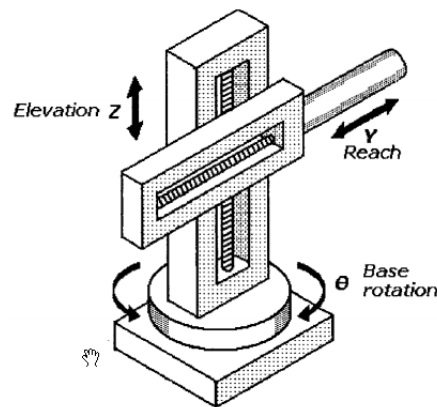


Figure 2.7: Cylindrical Robot arm diagram



Figure 2.8: Cylindrical Robot arm

2.3.3 Spherical or Polar Robot

Spherical robots, sometimes regarded as polar robots, are stationary robot arms with spherical or near-spherical work envelopes that can be positioned in a polar coordinate system.

So, these robots are more sophisticated than Cartesian and cylindrical robots, while control solutions are less complicated than those of articulated robot arms. This may be the reason why sometimes they are used as a base for robot kinematics exercises.

Also, it should be noted that spherical robot type takes a significant spot in robot history, as some of the first robot arms can be counted into this type. Modern industrial robot arms kind of evolved from spherical robots and are sometimes regarded as this type, as their work envelope often is sphere-like as well.

The spherical robots' structure is the one you can see in (Figure 2.9), it has two rotary joints and one linear. Thus a spherical work envelope is formed. Of course, it is not a sphere, but reachable places can still be calculated in a polar coordinate system.

However there can be more than three joints. However, these three are the basic ones that form the work envelope. Further joints would add more flexibility, but would not radically change the reachable area. For example, if the gripper could rotate, this would be a 4-axis robot while the work envelope would not change at all.

2.3.4 Articulated Robot

The articulate or jointed arm robot (or sometime called Anthropomorphic arms) closely resembles the human arm. The mechanical structure has at least three rotary joints which forms a polar coordinate system. The Figure 2.11 shows an articulate robot with 6 degree of freedom. The basic three

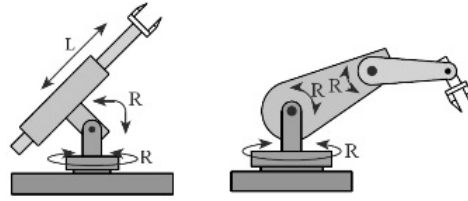


Figure 2.9: Spherical Robot arm diagram



Figure 2.10: Spherical Robot arm

rotary joints able Arm swap, shoulder swivel and elbow rotations. Additional 3 revolute joints (Roll, Yaw, Pitch) and one prismatic joint allow the robot to point in many directions, and then reach out some radial distance.

This structure is very flexible and has the ability to reach over obstructions. It can generally achieve any position and orientation within the working envelope. As such articulate robots are used for a wide range of applications including paint spraying, arc and spot welding, machine tending, etc. For examples, the articulate robot allows the welding torch to be manipulated in almost the same fashion as a human being would manipulate it. The torch angle and travel angle can be changed to make good quality welds in all positions. Articulate robots also allow the arc to weld in areas that are difficult to reach. In addition, articulate robots are compact and provide the largest work envelope relative to their size.

Typical articulate robots have five or six free programmable arms or axes. As mentioned, the flexibility of the articulate robots make them well suit for a wide variety of industrial application. But, it is not easy to control. When driving these robots in their natural co-ordinate system (joint space) the motion of the robot from one point to another can be difficult to visualize as the robot will move each joint through the minimum angle required. This means that the motion of the tool will not be a straight line.

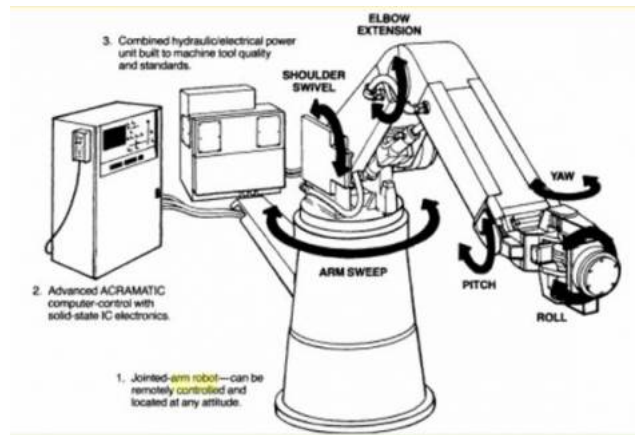


Figure 2.11: Articulated Robot arm diagram

2.4 Kinematics

2.4.1 Overview

Kinematics is the branch of classical mechanics that describes the motion of points, bodies and systems of bodies without consideration of the forces that cause it. It is also defined as the relationships between the positions, velocities, and accelerations of the links of the robot. There are two main types of kinematics, forward and inverse kinematics. (See Figure 2.12)

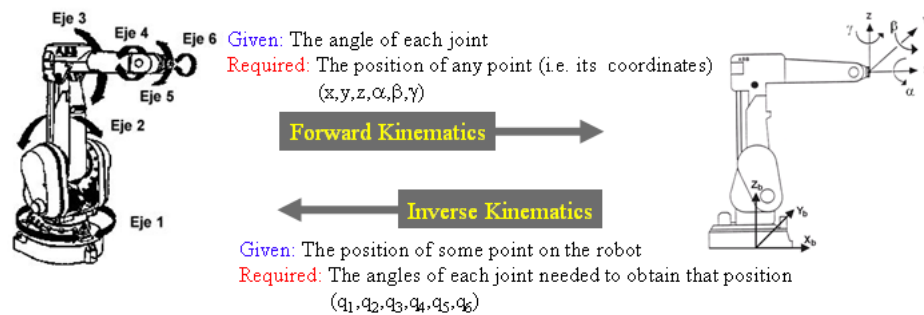


Figure 2.12: Types of Robot Kinematics

2.4.2 Kinematic Chains

A robot manipulator is composed of a set of links connected together by various joints. The joints can either be very simple, such as a re-volute joint or a prismatic joint, or else they can be more complex, such as a ball

and socket joint. The difference between the two situations is that, in the first instance, the joint has only a single degree-of-freedom of motion: the angle of rotation in the case of a re-volute joint, and the amount of linear displacement in the case of a prismatic joint. In contrast, a ball and socket joint has two degrees-of-freedom.

With the assumption that each joint has a single degree-of-freedom, the action of each joint can be described by a single real number: the angle of rotation in the case of a re-volute joint or the displacement in the case of a prismatic joint. The objective of forward kinematic analysis is to determine the cumulative effect of the entire set of joint variables.

A robot manipulator with n joints will have $n + 1$ links, since each joint connects two links. We number the joints from 1 to n , and we number the links from 0 to n starting from the base. By this convention, joint i connects link $i - 1$ to link i . We will consider the location of joint i to be fixed with respect to link $i - 1$. When joint i is actuated, link i moves. Therefore, link 0 (the first link) is fixed, and does not move when the joints are actuated.

With the i^{th} joint, we associate a joint variable, denoted by q_i is the angle of rotation, and in the case of a prismatic joint, q_i is the joint displacement:

$$q_i = \begin{cases} \theta_i & : \text{joint } i \text{ re-volute,} \\ d_i & : \text{joint } i \text{ prismatic.} \end{cases} \quad (2.1)$$

To perform the kinematic analysis, we rigidly attach a coordinate frame to each link. In particular, we attach $o_i x_i y_i z_i$ to link i . This means that, whatever motion the robot executes, the coordinates of each point on link i are constant when expressed in the i^{th} coordinate frame. Furthermore, when joint i is actuated, link i and its attached frame, $o_i x_i y_i z_i$, experience a resulting motion. The frame $o_0 x_0 y_0 z_0$, which is attached to the robot base, is referred to as the inertial frame. Equation 2.1 illustrates the idea of attaching frames rigidly to links in the case of an elbow manipulator.

A_i is the homogeneous transformation matrix that gives the position and orientation of $o_i x_i y_i z_i$ with respect to $o_0 x_0 y_0 z_0$. The matrix A_i is not constant, but its a function of the joint variable q_i . The homogeneous transformation matrix that expresses the position and orientation of $o_i x_i y_i z_i$ with respect to $o_j x_j y_j z_j$ is called a transformation matrix, and is denoted by T_j^i .

Then the position and orientation of the end effector in the inertial frame are given by $H = T_n^0$.

2.4.3 The Denavit-Hartenberg Convention

The Denavit-Hartenberg, or DH convention is a commonly used convention for selecting frames of reference in robotics applications. In this convention, each homogeneous transformation A_i is represented as a product of four basic transformations:

$$\begin{aligned}
 A_i &= Rot_{z,\theta_i} Trans_{z,d_i} Trans_{x,a_i} Rot_{x,\alpha_i} \\
 &= \begin{pmatrix} c_{\theta_i} & -s_{\theta_i} & 0 & 0 \\ s_{\theta_i} & c_{\theta_i} & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & d_i \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 & a_i \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & c_{\alpha_i} & -s_{\alpha_i} & 0 \\ 0 & s_{\alpha_i} & c_{\alpha_i} & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \\
 &= \begin{pmatrix} c_{\theta_i} & -s_{\theta_i}c_{\alpha_i} & s_{\theta_i}s_{\alpha_i} & a_ic_{\theta_i} \\ s_{\theta_i} & c_{\theta_i}c_{\alpha_i} & -c_{\theta_i}s_{\alpha_i} & a_is_{\theta_i} \\ 0 & s_{\alpha_i} & c_{\alpha_i} & d_i \\ 0 & 0 & 0 & 1 \end{pmatrix}
 \end{aligned} \tag{2.2}$$

where the four quantities $\theta_i, a_i, d_i, \alpha_i$ are parameters associated with link i and joint i . The four parameters a_i, α_i, d_i , and θ_i in the Equation (2.2) are generally given the names link length, link twist, link offset, and joint angle, respectively. These names derive from specific aspects of the geometric relationship between two coordinate frames. Since the matrix A_i is a function of a single variable, it turns out that three of the above four quantities are constant for a given link, while the fourth parameter, θ_i for a re-volute joint and d_i for a prismatic joint, is the joint variable.

2.4.4 Forward Kinematics

As mentioned above, the forward kinematics problem is the calculation the position and orientation of the end effector or tool in terms of the joint variables. We make extensive use of the DH convention in order to solve the forward kinematics problem. First, we calculate the DH parameters for the robot. This step implies drawing the coordinate frames of all the joints axes. Next, we calculate the A matrices mentioned above for each frame. All the matrices are then multiplied resulting in the final forward kinematic equation:

$$T_n^0 = A_1 \dots A_n$$

2.4.5 Inverse Kinematics

The inverse is the problem of finding the joint variables in terms of the end-effector position and orientation. In general, it is more difficult than the forward kinematics problem. Going to the General Inverse Kinematics problem, given a 4x4 homogeneous transformation:

$$H = \begin{bmatrix} R & 0 \\ 0 & 1 \end{bmatrix} \in SE(3) \quad (2.3)$$

with $R \in SO(3)$, find (one or all) solutions for the equation

$$T_n^0(q_1, \dots, q_n) = H \quad (2.4)$$

where

$$T_n^0(q_1, \dots, q_n) = A_1(q_1) \dots A_n(q_n). \quad (2.5)$$

Here, H represents the desired position and orientation of the end-effector, and our task is to find the values for the joint variables $q_1, \dots, q_n$ so that $T_n(q_1, \dots, q_n) = H$.

Equation (2.4) results in twelve nonlinear equations in n unknown variables, which can be written as

$$T_{ij}(q_1, \dots, q_n) = h_{ij}, i = 1, 2, 3, j = 1, \dots, 4 \quad (2.6)$$

where T_{ij} , h_{ij} refer to the twelve nontrivial entries of T and H , respectively.

Although the general problem of inverse kinematics is quite difficult, it turns out that for manipulators having six joints, with the last three joints intersecting at a point, can be decoupled using kinematic decoupling approach. So the main idea of decoupling the inverse kinematic problem is to represent it into two simpler problems, known respectively, as inverse position kinematics, and inverse orientation kinematics. Mainly it is done by finding the position of the wrist axes, then finding the orientation of the wrist.

In general, the complexity of the inverse kinematic problems increases whenever the number of non-zero DH parameters increases. In some situations, the inverse kinematic problem can not have a solution due to the non-reachability of the end-effector. Furthermore, if a solution found, it must later be checked for joint limits because in some cases the solution can be out of the robot's joint range.

2.5 Challenging problems

Most robots today can trace their origin to early industrial robot designs. Much of the technology that makes robots more human-friendly and adaptable for different applications has emerged from manufacturers of industrial robots. Industrial robots are by far the largest commercial application of robotics technology today. All the important foundations for robot control were initially developed with industrial applications in mind. These applications deserve special attention in order to understand the origin of robotics science and to appreciate many unsolved problems that still prevent the wider use of robots in manufacturing. In this section we present a brief description about the challenging problems that confront the industrial robot arm manufacturing.

2.5.1 Environment perception

The definition of robotics perception is to enable the robot to combine data from the sensors in order to create combined, purposeful perception. Generally, perception consists of several levels, one of these levels is lifelong mapping and environment reconstruction. An example of mapping systems is to take one sweep through an environment, construct a map off-line, and then use it until the environment changes enough to require another sweep. So the problem here is to continuously update a detailed geometrical map as the robot goes about its tasks, using both vision and laser sensors to recognize places, and label spaces and objects therein.

Another perception focus is reliable object detection, registration, and tracking. In addition to manipulating a tabletop objects. Therefore, a real-time recognition of common small objects is considered from the robotics perception problems. There exists other problems about perception which consist of detecting people, understanding their actions, tracking face pose and their body motion.

Currently, computer vision is good at recognizing objects in images and videos and has been successfully employed in several specialized industrial applications, such as quality control during microchip fabrication. But robotic perception is much weaker in less defined situations, like understanding and responding to human behavior and even conversations. Yet, it is precisely this sort of interaction which promises the most compelling applications for future humanoid technology, where people-like robots can act as guides, or mix with people, or even use perception to infer appropriate actions.

More importantly, these broad robotic applications will deliver insights into other disciplines, like cognition and neuro-science. A truly perceptive robot, capable of acting independently and appropriately in complex situations remains a distant goal.

2.5.2 Path planning

Autonomous robots which work without human operators are required in robotic fields. In order to achieve tasks, autonomous robots have to be intelligent and should decide their own actions. When the autonomous robot decides its action, it is necessary to plan optimally depending on their tasks. More, it is necessary to plan a collision free path minimizing a cost such as time, energy and distance. When an autonomous robot moves from a point to a target point in its given environment, it is necessary to plan an optimal or feasible path avoiding obstacles in its way and answer to some criterion of autonomy requirements such as : thermal, energy, time, and safety for example. Therefore, the major main work for path planning for autonomous mobile robot is to search a collision free path.

Many works on this topic have been carried out for the path planning of autonomous mobile robot. Motion planning is one of the important tasks in intelligent control of an autonomous mobile robot. It is often decomposed into path planning and trajectory planning. Path planning is to generate a collision free path in an environment with obstacles and optimize it with respect to some criterion. Trajectory planning is to schedule the movement of a mobile robot along the planned path. Several approaches have been proposed to address the problem of motion planning of a mobile robot. If the environment is a known static terrain and it generates a path in advance it said to be off-line algorithm. It is said to be on-line if it is capable of producing a new path in response to environmental changes.

So simply, robot path planning is about finding a collision-free motion from one position to another. Efficient algorithms for solving problems of this type have important applications in areas such as: industrial robotics, computer animation, drug design, and automated surveillance. It is therefore not surprising that the research activity in this field has been steadily increasing over the last two decades.

2.5.3 Object grasping

Modern-day robots can be carefully hand-programmed or scripted to carry out many complex manipulation tasks, ranging from using tools to assemble complex machinery, to balancing a spinning top on the edge of a sword. However, autonomously grasping a previously unknown object still remains a challenging problem. If the object was previously known, or if we are able to obtain a full 3-D model of it, then various approaches, for example ones based on friction cones, form- and force-closure, pre-stored primitives, or other methods can be applied.

Some approaches have been made and most of the previous works have been heavily relying on the 3-D model of an object to successfully grasp it. However, building the 3-D model of a newly seen object itself is a very challenging task. To address this difficulty, proposes the learning algorithm that does not require building the 3-D model of an object. Rather, they utilize the features extracted from 2-D images to predict possible grasping points of the object. They demonstrate that a robot can successfully grasp the object that it never saw before, but use only the image data of an object leading to limited performance. Another proposed system is to use the depth map of an object in addition to the image to improve performance.

Finally, even if more specialized sensors such as laser scanners (or active stereo) are used to estimate the object's shape, we would still have only a 3-D reconstruction of the front face of the object. So robotics object grasping still confront a problem for the developers.

Chapter 3

Simulation Environment for Industrial Robots: A Review

3.1 Introduction

As mentioned before, a robotics simulator is used to create embedded applications for a robot without depending physically on the actual machine, thus saving cost and time. In some case, these applications can be transferred on the real robot (or rebuilt without modifications. The term robotics simulator can refer to several different robotics simulation applications. The most popular application for robotics simulators is for 3-D modeling and rendering of a robot and its environment. This type of robotics software has a simulator that is a virtual robot, which is capable of emulating the motion of an actual robot in a real work envelope. Even though, for more realistic motion generation, sine robotics simulators use a physics engine.

Regardless the existence and the availability of an actual robot or not, the use of robotics simulator for development of a robotics control program is highly recommended. The simulator allows for robotics programs to be conveniently written and debugged off-line with the final version of the program tested on an actual robot. Of course, this primarily holds for industrial robotic applications only, since the success of off-line programming depends on how similar the real environment of the robot to the simulated environment. Sensor-based robot actions are much more difficult to simulate and/or to program off-line, since the robot motion depends on the instantaneous sensor readings in the real world.

3.2 Simulation Environment Requirements

This section is talking about the simulation system requirements, in other words, the minimum requirements which a system or a program should have in order to become a robotics simulation system. So we have two main classes or two main subsections, the first one that we will mention is the system requirements, and the other one is the user requirements.

For the first part, we will mention the requirements that any base system should provide in order to let the developers integrate their robotics simulation with. Basically, one of these requirements is the ability of interacting with the base system using a known scripting language, for example C++ or Python. Besides, the base system should have the ability to display a two or three dimensions environment, the ability to load at least one known robot into that environment, and letting the loaded robot to interact with the other surrounding objects.

Moreover, the base system should provide some basic functions, and the developers could understand and modify easily in order to achieve their goals from the system. Example of these functions is a method that changes the loaded robot-arm's joints angle. An inverse kinematics method which has a role of solving the inverse kinematics matrix for a given body with a specific location should be also one of these functions. Besides, the system should maintain a function which could enable the developers, as well as the users, to place some objects in the 2-D or the 3-D environment. Another option which puts raises the base system to the professional layer is that system is based on a strong physics engine.

On the other hand, for the user requirements which are considered mainly as the developers's objective, it should contain a user interface that has a role of informing the user about the current system's situation, and also letting the user to interact with the base system through the user interface, like it shown in the proposed system. Mainly, the user interface should enable the user to control the environment as well as the robot itself, and also inform the user what he/she should do to execute a specific command like changing a specific joint angle, execute the forward and inverse kinematics functions, and also inform the user what are the commands passed to the system. So generally, these are the basic requirements that a robotic simulation system should maintain.

3.3 Survey on Simulation Environment

In this section, a survey about some known 3-D robotics simulation systems and programs will be provided and discussed. So the systems that will be discussed in details are: Robot Operating system (ROS), Gazebo, Webots, Blender for robotics known as Morse, Simbad 3-D Robot Simulator, Open Robotics Automation Virtual Environment known as OpenRave and finally the Microsoft Robotics Developer Studio system.

Robot Operating System

ROS is a software framework for robot software development, providing operating system-like functionality on a heterogeneous computer cluster. "Switch-yard" was originally the name of ROS when it was created and developed in 2007 by the Stanford Artificial Intelligence (AI) Laboratory in support of the Stanford AI Robot project. Mainly, ROS provides standard operating system services such as hardware abstraction, low-level device control, implementation of commonly-used functionality, message-passing between processes, and package management. It is based on a graph architecture where processing takes place in nodes that may receive, post and multiplex sensor, control, state, planning, actuator and other messages.

For Ubuntu, Linux and the Unix-like system, ROS library is geared and supported perfectly. On the other hand, other variants such as Fedora and Mac OS X are still considered as experimental. Generally, ROS has two basic "sides": The operating system side ROS as described above and ROS-pkg, which consists of a suite of user contributed packages, organized into sets called stacks, and implement functionality such as simultaneous localization and mapping, planning, perception, simulation etc. Technically, ROS areas include a master coordination node, publishing or subscribing to data streams like images, stereo, laser, control, etc; multiplexing information, node creation and destruction, nodes are seamlessly distributed, allowing distributed operation over multi-core, multi-processor, GPUs and clusters, logging, parameter server, and test systems. Furthermore, ROS Package application areas include environment and object perception, object Identification, segmentation and recognition, face recognition, gesture recognition, motion tracking, Ego-motion, motion understanding, structure from motion, stereo vision which consists of depth perception via two cameras, motion, mobile robotics, control, planning and grasping. ROS is released under the terms of the BSD license, and is open source software. It is free for commercial and research use. The ROS-pkg contributed packages are licensed under

a variety of open source licenses.

3.3.1 Gazebo

Gazebo has started in the fall of 2002 at the University of Southern California, with the concept of a high-fidelity simulator stemmed from the need to simulate robots in outdoor environments under various conditions. Gazebo was originally designed to aid in the development process of algorithms for robotic platforms. By realistically simulating robots and environments code designed to operate a physical robot can be executed on an artificial version. This helps to avoid common problems associated with hardware such as short battery life, hardware failures, and unexpected and dangerous behaviors. It is also much faster to spin up a simulation engine than continually run code on robot, especially when the simulation engine can run faster than real-time.

Over the years Gazebo has also been used for regression testing. Scenarios designed to test algorithm functionality have been established and passed through test rigs. These tests can be run continually to maintain code quality and functionality. Numerous researchers have also used Gazebo to develop and run experiments solely in a simulated environment. Controlled experimental setups can easily be created in which subjects can interact with robots in a realistic manner. Additionally, Gazebo has been used to compare algorithms for such things as navigation and grasping in a controlled environment.

Therefore, Gazebo is considered as a multi-robot simulator for outdoor environments. Also, it is capable of simulating a population of robots, sensors and objects, but it does so in a three-dimensional world. Moreover, it generates both realistic sensor feedback and physically plausible interactions between objects, therefore it includes an accurate simulation of rigid-body physics. Also Gazebo can be integrated and run with Robot operating System (ROS) that was mentioned before. Moreover, it is under active development at Willow Garage.

3.3.2 Webots

Webots is a development environment used to model, program and simulate mobile robots. With Webots the user can design complex robotic setups, with one or several, similar or different robots, in a shared environment. The properties of each object, such as shape, color, texture, mass, friction, etc.,

are chosen by the user. A large choice of simulated sensors and actuators is available to equip each robot. The robot controllers can be programmed with the built-in IDE or with third party development environments. The robot behavior can be tested in physically realistic worlds. The controller programs can optionally be transferred to commercially available real robots. Webots is used by over 750 universities and research centers worldwide. With Webots, you take advantage of a proven technology that has been co-developed by the Swiss Federal Institute of Technology in Lausanne, thoroughly tested, well documented and continuously maintained for over 10 years.

The philosophy behind Webots is that mobile robotics is a complex research area in which many advanced technologies and open research issues are combined all together. Technologies include electronics, mechanics, computer hardware and software. It is often difficult to master perfectly every technology, and hence mobile robotics projects require an important investment. Realistic simulations and fast prototyping of mobile robots help reducing the amount of time and hardware spent in developing mobile robotics applications. Moreover, such software tools allow researchers and students to focus on the most interesting parts of their robotics projects and hence achieve more advanced results.

One of Webots's features that makes it a unique application in robotics simulation is that Webots is portable robot simulator, therefore it runs natively on Windows, Mac and Linux. Also, both world files and API functions are cross-platform, which means that they can easily be shared by people using different operating systems. Furthermore, it creates dynamic worlds simulated with the Open Dynamics Engine (ODE). So the user could specify the bounds, the mass, the center of mass, and the friction coefficients of your objects, and let Webots simulate their dynamic behavior. Another feature is the user could specify also the collision bounds of each object, independently from its graphical representation, and use any combination of the following primitives: Box, Sphere, Capsule, Cylinder, Indexed-Face-Set, Elevation-Grid or Plane. In addition, Webots file format is human-readable, fully documented and can be modified with regular text editors. So the user can design his own tools to automatically generate the Webots worlds that suits his needs.

Also, the simulation using Webots can have as many robots as the user needs. The memory and the CPU power of the computer are the only limits. Each robot may be a different model with its own specific controller code

written in any programming language. For instance, the developer can program the robots using 200 API functions available in 6 different languages: C, C++, Java, Python, Matlab, URBI. He also could use object-oriented APIs with 20 classes in C++, Java or Python, and even link with external libraries, e.g., OpenCV. Using Matlab, the robots controllers can be programmed as .m files by using the 200 functions of the Matlab API (Matlab license required). Use Matlab's Image Processing Toolbox to process Webots camera images is also from Webots features. Like Gazebo, Webots could connect to ROS using the provided roscpp (C++) or the rospy (Python) controller interfaces and take advantage of all ROS stacks, including OpenCV, OpenRAVE, OpenSlam GMapping, inverse kinematic, etc.

Get programming access to the ODE-based physics simulation engine to define advanced physics behaviors (e.g., add fluid dynamics, non-uniform friction, etc.) or simulate new sensors or actuators are provided also by Webots. In addition to watching and interacting with the running simulation in the 3-D simulation window. Webots's user can easily drag, rotate, lift objects at any time, even when the simulation is running. This is useful to test the robustness of your robots to unexpected events. Create screen shots and record high quality videos directly from Webots menus is also provided. Create realistic simulation models with advanced lighting, fog, shadows, textures and transparency is also an option in Webots.

The user can take benefit from the large collection of commercial robot models included in Webots: Aibos, Bioloids, Boe-Bot, e-puck, HOAP-2, iRobot Create, Katana, Kheperas, Koala, Kondo KHRs, Nao, Pioneer, Shrimp III, Surveyor SVR-1, etc and tries the collection of ready-to-use interactive objects: walls, doors, windows, lights, chairs, sofa, tables, bookcase, lockers, television, fridge, oven, soda can, fruits, books, plants, paintings, car, rocks, barrel, soccer ball, etc. Besides, the user could equip his simulated robots with customizable and accurate sensors models from the provided sensor library: distance sensor, light sensor, cameras, LIDARs, GPS, gyrometer, accelerometer, compass, bumpers, force-sensor, etc.

3.3.3 Modular Open Robots Simulation Engine

The Modular Open Robots Simulation Engine (MORSE) is an generic simulator for academic robotics. It focuses on realistic simulation of small to large environments, indoor or outdoor, with one to over a dozen of autonomous robots. Also MORSE provides out of the box a set of standard

sensors (cameras, laser scanner, GPS, odometry, etc.), actuators (speed controllers, high-level waypoints controllers, generic joint controllers) and robotic bases (ATRV, generic 4 wheel vehicle, PR2,...), new ones can easily be added. One of the main design choice for MORSE is the ability to control the degree of realism of the simulation: if the user is working on vision, he needs accurate camera sensors, but may not care about the realism of the motion controller, and he may find a way-point controller good enough and even easier to use. On the contrary, if the user works on robot supervision, he may prefer skip the perception stack and directly work with objects ID and positions. Moreover, MORSE lets the developer to define how realistic the different components of the robot need to be to fit the developer's needs.

MORSE has an original work flow: since it uses Blender and the Blender Game Engine as runtime, it benefits from the tight integration of these two tools. Therefore, The user builds his simulation in Blender by assembling together components to create one or several robots (like an empty ATRV robot base with a camera, a laser scanner and a waypoints actuator - once complete, the user can of course save it) and an environment (like an indoor lab room with tables and chairs, or an outdoor environment with a village, etc. A couple of them are provided to start with). Then, switching forth and back from the edit mode (where the robot and 3-D environment are build) to the simulation itself is instantaneous. Debugging or tuning the simulation scenario is also a breeze. And for people who do not like too much GUIs, MORSE also provides a complete Python API to programatically create the environments and robots, and run the simulation from the command line. Once started, robots, sensors, actuators are accessible from outside through the usual middleware.

MORSE do not make any assumption on the architecture. MORSE currently supports out of the box 4 common open-source middle-wares (ROS, YARP, Pocolibs and MOOS) plus offers a generic socket-based communication channel. It also offers a large set of standard sensors and actuators. To even better adapt to the user's specific need, MORSE provides a lightweighted overlay mechanism to quickly change the name and types of exchanged data flows. Morse developer should hopefully be able to integrate MORSE in his own architecture without any modifications to his existing components. Lastly, MORSE benefits from Blender import/export capabilities: existing models in many 3-D formats (Collada, DXF, 3DS Max, VRML, etc.) can be used to build robots and environments.

Mainly, the system rendering is based on the Blender Game Engine. The OpenGL-based Game Engine supports shaders, provides advanced lightning options, supports multi-texturing, and use the state-of-the-art Bullet library for physics simulation. Therefore, enabling MORSE users to create rich simulation environments, visually and physically plausible. Besides, the program has also been designed to be able to handle over a dozen of robots in joint simulation scenario, to this end, MORSE can be run as a distributed network of simulation nodes. Each node automatically synchronizes with the others. However, due to latencies, do not expect to simulate accurate physical interactions in the distributed mode.

While designed as a generic simulation software, MORSE features several tools dedicated to human-robot interaction (HRI) simulation. MORSE provides a complete human rig, that can be controlled in first-person perspective. In this mode, the user can look around, pick and place objects, move them, etc. The robot can access in return to the full human posture which is similar to the Kinect output. Generally, MORSE is mostly written in Python, except for computation intensive processes (like 3-D rendering or physics simulation), MORSE is a purely Python application. This enable easy and fast modification of the source code. Besides, MORSE has been designed from the beginning to be modular: adding a new sensor, a new actuator, a post-processing (like applying a noise function), adding new services, or even a complete communication middle-ware is easy and documented.

Unlike numerous open-source projects, MORSE has a complete and up-to-date online documentation, both for the users and the developers. Finally, MORSE and all the libraries it relies on are open-source projects, this means that anyone can not only use it at no cost, but also access to the source code to modify it if you wish so. MORSE itself is licensed under a permissive BSD license and this also means that MORSE follows an open development process.

3.3.4 Simbad

Simbad is a Java 3d robot simulator for scientific and educationnal purposes. It is mainly dedicated to researchers/programmers who want a simple basis for studying Situated Artificial Intelligence, Machine Learning, and more generally AI algorithms, in the context of Autonomous Robotics and Autonomous Agents. It is not intended to provide a real world simulation and is kept voluntarily readable and simple. It also enables programmers

to write their own robot controller, modify the environment and use the available sensors. The Simbad project is hosted at SourceForge, the Simbad simulator is also an open source which means that Simbad simulator is free for anyone to use and modify under the conditions of the GNU General Public Licence.

Generally, the main features of Simbad are providing 3-D visualisation and sensing, single or Multi-robots simulation, vision Sensors by color monoscopic camera, range Sensors with sonars and IR, Contact Sensor bumpers and swing User interface for control.

3.3.5 Microsoft Robotics Developer Studio

Microsoft Robotics Developer Studio 4 (RDS 4) provides a wide range of support to help make it easy to develop robot applications. RDS 4 includes a programming model that helps make it easy to develop asynchronous, state-driven applications. RDS 4 provides a common programming framework that can be applied to support a wide variety of robots, enabling code and skill transfer. RDS 4 also includes a lightweight asynchronous services-oriented runtime, a set of visual authoring and simulation tools, as well as templates, tutorials, and sample code to help you get started.

Furthermore, Concurrency and Coordination Runtime (CCR) helps make it easier to handle asynchronous input and output by eliminating the conventional complexities of manual threading, locks, and semaphores. Lightweight state-oriented Decentralized Software Services (DSS) framework enables developers to create program modules that can inter-operate on a robot and connected computers by using a relatively simple, open protocol. Besides, Visual Programming Language (VPL) provides a relatively simple drag-and-drop visual programming language tool that helps make it easy to create robotics applications. VPL also provides the ability to take a collection of connected blocks and reuse them as a single block elsewhere in your program. VPL is also capable of generating human-readable C#.

DSS Manifest Editor (DSSME) provides a relatively simple creation of application configuration and distribution scenarios. Visual Simulation Environment (VSE) provides the ability to simulate and test robotic applications using a 3-D physics-based simulation tool. This allows developers to create robotics applications without the hardware. Sample simulation models and

environments enable you to test your application in a variety of 3-D virtual environments.

Open Robotics Automation Virtual Environment

OpenRAVE was founded at the Quality of Life Technology Center in the Carnegie Mellon University Robotics Institute. It was inspired from the RAVE simulator James Kuffner had started developing in 1995 and used for his experiments ever since. The OpenRAVE project was started in 2006 and is a complete rewrite of RAVE. It is actively being maintained at the JSK Lab at University of Tokyo.

Nowadays OpenRAVE includes a seamless integration of simulation, visualization, planning, scripting and control. The plugin architecture allows users to easily write custom controllers or extend functionality. Using OpenRAVE plugins, any planning algorithm, robot control, or sensing-based subsystem can be distributed and dynamically loaded at run-time; this distributed nature frees developers from struggling with monolithic code-bases. Users of OpenRAVE can concentrate on the development of planning and scripting aspects of a problem without having to explicitly manage the details of robot kinematics and dynamics, collision detection, world updates, and robot control. OpenRAVE provides a powerful Python API for scripting demos, which makes it simple to control and monitor the demo and environment state. There are also interfaces for Octave and Matlab.

OpenRAVE's major design goals and features are: have a plugin-based architecture that allows users to expand its functionality without having to recompile the base code. Most functionality should be offered as plugins, thus keeping the core as simple as possible. It also offer many motion planning algorithm implementations that can be easily extended to new tasks. Make it easy to debug components during run-time without having to recompile or restart the entire system in order to prevent flushing of the in-memory environment state. Allow the OpenRAVE core to be used as a simulation environment, as a high-level scripting environment, as a kinematics and dynamics backend module for robot controllers, or as a manipulation planning black box in a distributed robotics environment. Allow simple planning knowledgebases to be generated, stored, and retrieved. Support a multi-threaded environment and allow easy parallelization of planners and other functions with minimal synchronization required on the user side.

One of OpenRAVEs strongest points when compared with other planning packages is the idea of being able to apply algorithms to any scenario with very little modification when robots or target objects change. Users of OpenRAVE can concentrate on the development of planning and scripting aspects of a problem without having to explicitly manage the details of robot kinematics, dynamics, collision detection, world updates, sensor modeling, and robot control.

OpenRAVE has been used for planning on many real robotics systems. Its architecture makes it possible for planning-enabled robots to work consistently in a continuously changing and unpredictable environment. Many new layers of functionality have been developed that go beyond the basic kinematics, collision detection, and graphics interface requirements of classic robotics libraries. It provides a set of interfaces that let users modify existing functions and expand OpenRAVE-enabled modules without having to recompile OpenRAVE or deal with messy monolithic code-bases.

3.4 Comparative Study

	ROS	Gazebo	Webots	Morse	Simbad	OpenRAVE	MRDS
OS	Linux, Mac, Win.	Linux.	Linux, Mac, Win.	Linux	Linux, Mac, Win.	Linux, Win.	Win.
Simulator Type	2-D, 3- D	3-D	3-D	3-D	3-D	3-D	3-D
Programming Language	C++, Python, Octave, LISP, Java, Lua	C, C++, Python, Java	C, C++, Java, Matlab, Python	Python	Java	Python, C++	VPL, C#, Visual Basic, JScript, Iron- Python
Documentation	High- Level	Low- Level	High- Level	High- Level	High- Level	High-Level	High- Level
Tutorial	Yes	Yes	Yes	Yes	Limited	Yes	Yes
Portability	Yes	Yes	Yes	Yes	Limited	Yes	Yes
Sensors	odometry, camera, range	odometry, camera, range	odometry, camera, range, GPS	camera, laser, scanner, GPS, odome- try	vision, range, contact	odometry, camera, laser	odometry, camera, range
Debugging/ Logging	No	Yes	Yes	Yes	Yes	Yes	Yes
Graphical User Interface	No	No	Yes	No	No	No	Yes
Open Source	Yes	Yes	No	Yes	Yes	Yes	No

Chapter 4

Proposed System

4.1 Introduction

In this chapter, we will focus more on the proposed system software and the implementation itself. Also a system block diagram has been designed for the proposed system and will be discussed. Moreover, this chapter covers a detailed explanation for each entity and component in the system.

4.2 System Block Diagram

4.2.1 Open Robotics Automation Virtual Environment

OpenRAVE provides an environment for testing, developing, and deploying motion planning algorithms in real-world robotics applications. The main focus is on simulation and analysis of kinematic and geometric information related to motion planning. OpenRAVE's stand-alone nature allows it to be easily integrated into existing robotics systems. An important target

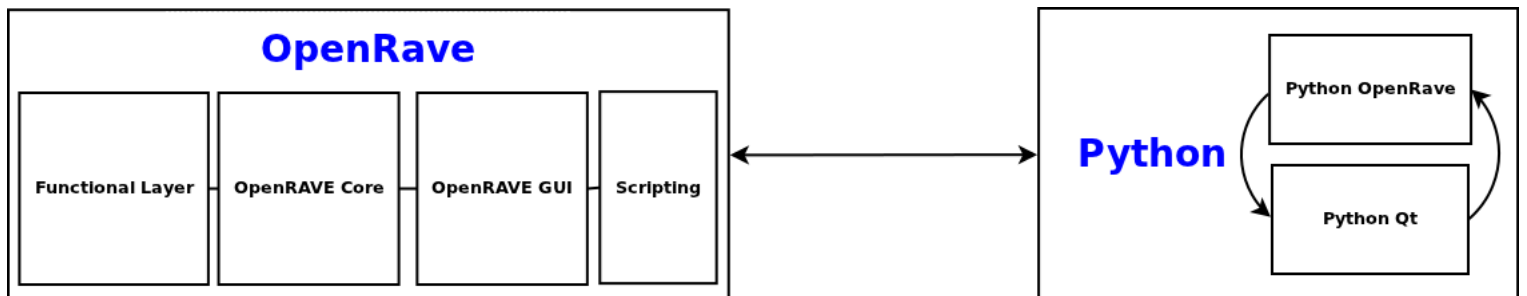


Figure 4.1: Proposed System Block Diagram

application is industrial robotics automation.

Concentrating on the development of planning and scripting aspects of a specific problem without having to explicitly manage the details of robot kinematics and dynamics, collision detection, world updates, sensor modeling, and robot control is the main goal that OpenRave tries to provide for users and developers. In addition, OpenRave provides the developers the capability to apply algorithms to any scenario with very little modification when robots or target objects change. Furthermore, it provides a powerful Python Application Programming Interface (API) for scripting demos which makes it simple to control and monitor the demo and the environment state. Also interfaces for Octave and Matlab have been provided by OpenRave.

OpenRAVE has been used for planning on many real robotics systems. Its architecture makes it possible for planning-enabled robots to work consistently in a continuously changing and unpredictable environment. Many new layer of functionality have been developed that go beyond the basic kinematics, collision detection, and graphics interface requirements of classic robotics libraries. It provides a set of interfaces that let users modify existing functions and expand OpenRAVE-enabled modules without having to recompile OpenRAVE or deal with messy monolithic code-bases.

The OpenRAVE architecture shown in Figure 4.1 simplifies the implementation and testing of planning algorithms and allows new modules to easily inter-operate with other modules through a well-defined API. The structure of operating systems, computer languages, and debugging tools has motivated the division of OpenRAVE into four major layers: a core layer for the API, a plugins layer for extending functionality, a scripting layer for runtime analysis, and a database layer for generation and quick retrieval of the knowledge-bases. These layers combine into a set of tools non-experts can use in analyzing their problem without requiring an in-depth knowledge of software systems, manipulation planning, and physics.

The OpenRAVE architecture began as a result of this thesis work and has evolved through the testing of many robot platforms and the feedback of a rapidly growing OpenRAVE community. Having a large user base composed of planning researchers and non-experts like mechanical engineers has provided invaluable feedback into how a manipulation framework should be designed and how the user should interact with it. As a consequence, OpenRAVE has gone through a re-design phase two times, each new design has automated

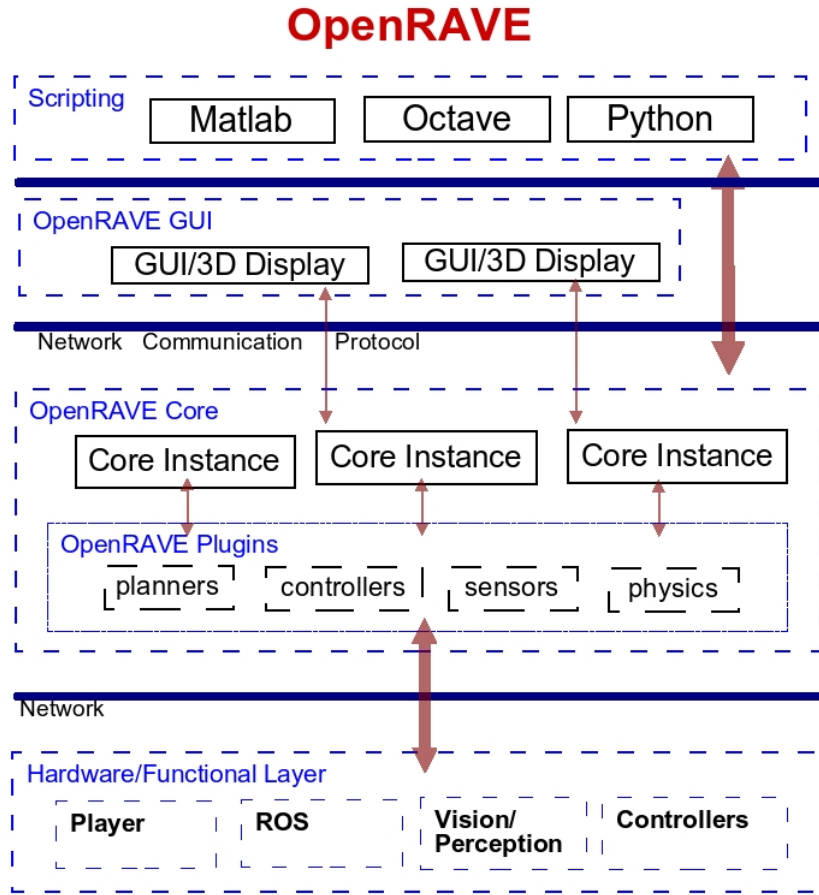


Figure 4.2: The OpenRAVE Architecture is composed of four major layers and is designed to be used in four different ways

more basic manipulation functions while removing functions that spread the environment too thin and do not provide much value. Such a process has allowed OpenRAVE to concentrate only in manipulation and geometric analyses, which has allowed new innovations beyond any capabilities of existing planning environments.

Generally, OpenRave provides some virtual environments in which robot arm can exist. All the environments used with OpenRave are represented in the XML language. There are about 55 different environments predefined in OpenRave system, furthermore, a user or a developer can create and implement his own 3-D environment and load it in OpenRave. The environment loaded by OpenRave combines collision checkers, viewers physics engines, the kinematic world, and all its interfaces into a coherent robotics world state.

Collision checking and detection, loading scenes and objects, managing objects and triangulation, drawing and plotting are the main functionalities of the environment classes in OpenRave. Figure 4.2 shows the predefined environment which has been selected to be loaded in the proposed system.

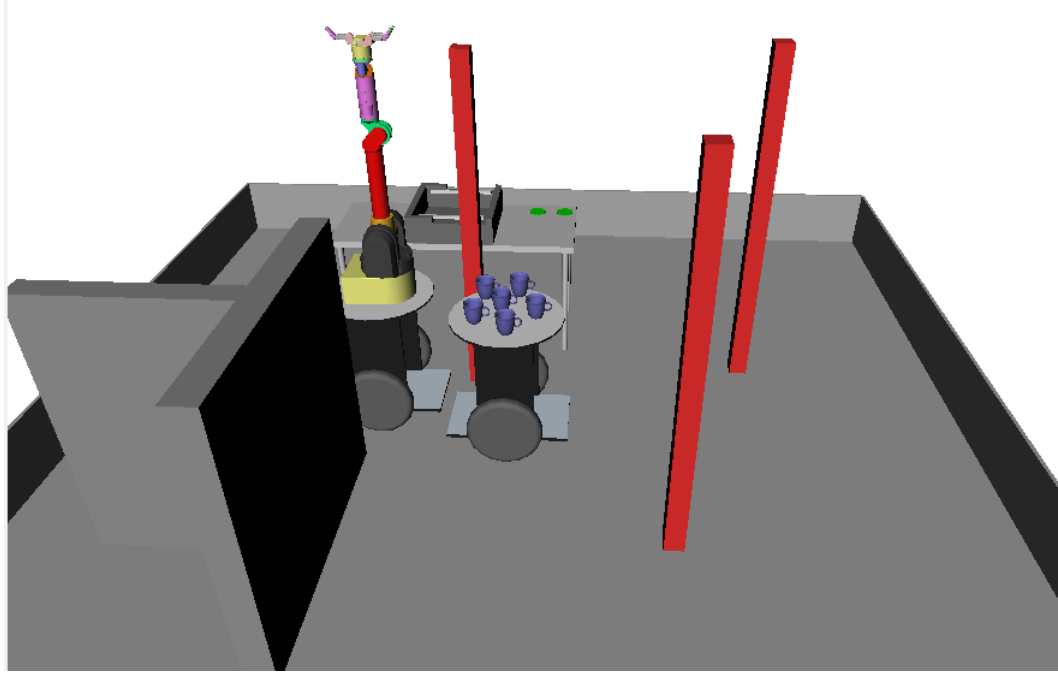


Figure 4.3: The proposed system environment

Also a Wam robot arm is loaded in the environment and like it is shown in Figures 4.3 and 4.4, the Barrett-Wam is a 7 degree-of-freedom robot arm (7-DOF) which consists of a shoulder, elbow and wrist. Furthermore, a gripper or a Barrett-Hand is attached to the end of the robot arm helping the robot arm to pick and place the objects.

The real WAM Arm is a highly dexterous, naturally back-drivable manipulator which stands for Whole Arm Manipulation and it is based on Puck electronics and mechanical drive technologies and designed to interact directly with people.

4.2.2 Python Programming Language

Python is a simple and minimalistic coding language. Reading a good Python program feels almost like reading English, although very strict English! This pseudo-code nature of Python is one of its greatest strengths.

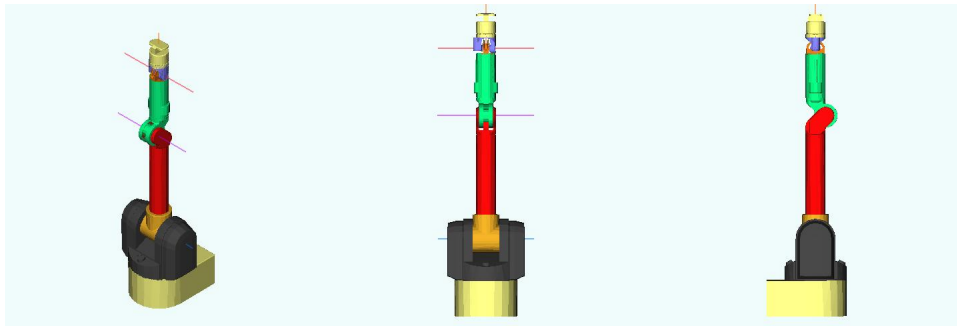


Figure 4.4: Proposed Barrett-Wam robot arm



Figure 4.5: Real Barrett-Wam robot arm

It allows you to concentrate on the solution to the problem rather than the language itself. Furthermore, procedure-oriented programming and object-oriented programming are supported by Python. In object-oriented languages, the program is built around objects which combine data and functionality. However in procedure-oriented languages, the program is built around procedures or functions which are nothing but reusable pieces of programs. So Python has a very powerful but simplistic way of doing object-oriented programming, especially when compared to big languages like C++ or Java.

Another feature is that Python is a high-level language, so the coder won't bother himself in managing the memory used by the program and other low-level details. In other languages such as C/C++, after running a certain program, a compiler is used to convert the C/C++ code into binary code in order that the processor understands the code. Moreover a linker/loader software is used to copy the program from hard disk to memory and the processor starts reading and executing the code. But Python has a different situation, after running the program, Python converts the source code into an

intermediate form called Bytecodes and then translates this into the native language of the computer and then runs it. Actually, all this makes using Python much easier since the coder does not have to bother himself with compiling the program or making sure that the proper libraries are linked and loaded.

To be able to hide part of the algorithm or running a piece of code very fast, the user can write that part of the program in C or C++ and then use them from Python program, and that makes Python an extensible language. Also the coder can embed Python within other C/C++ programs to give scripting capabilities for the program's users. We could describe the Python Standard Library as a huge indeed library, because it could help in doing various things involving regular expressions, documentation generation, unit testing, threading, databases, web browsers, CGI, ftp, email, XML, XML-RPC, HTML, WAV files, cryptography, GUI (graphical user interfaces), Tk, and other system-dependent stuff. Remember that all this is always available wherever Python is installed. This is called the 'Batteries Included' philosophy of Python. Besides, the standard library, there are various other high-quality libraries such as WXPYthon, Twisted, Python Imaging Library and many more.

Another feature that makes Python a powerful programming language is that Python is free and open source, in simple terms, anybody can freely distribute copies of this software, read its source code, make changes to it and use pieces of it in new free programs. Also Python can be ported and used on many platforms such as Linux, Windows, FreeBSD, Macintosh, Solaris, OS/2, Amiga, AROS, AS/400, BeOS, OS/390, z/OS, Palm OS, QNX, VMS, Psion, Acorn RISC OS, VxWorks, PlayStation, Sharp Zaurus, Windows CE and even PocketPC without requiring any changes due to its open-source nature.

Focusing on the proposed system, there are two main Python programs. The role of the first program is to connect and deal with OpenRave server and libraries. Also it is used as an application programming interface for OpenRave as declared before. The second Python program is for drawing and displaying the User Interface (UI) that has been generated using PYQT. Finally the two Python programs connect together to let the user pass commands to OpenRave through the user interface.

Python with OpenRave

In detail, a control server class 'Server()' has been implemented that has a role of running the benchmark in its own process, also it initializes 2 pipelines, the first one for OpenRave control server and the other one for a general server, in order to keep them running simultaneously. Moreover, the control server class executes 'StartQtGuiControl()' function which has a task of running the PYQT program, the program which responsible for displaying and controlling the graphical user interface and we will mention that later in detail. Eventually, this class creates the main server's loop which waits and listens for any input or signal coming from the PYQT program.

Equally important, the 'OpenRaveServer()' class, which has a mission of handling the commands passed to OpenRave. For example, this command could be load the required environment or control the robot itself, or even more, it could be calling a specific function from OpenRave. To explain more, this class loads the required environment, which has been described in Chapter 2, also this class contains several methods and functions, each one has a specific role and they run based on the appropriate passed command. In particular, OpenRaveServer class contains a direct control method which control the robot and could move each joint that belongs to this robot. Another method exists in the same class, the inverse kinematics function, which takes a double array as a parameter, creates a "kinbody" and translates this kinbody with the required x, y and z values that were passed in the array. Also this function runs a predefined method in OpenRave to solve the inverse kinematics matrix for the kinbody, and consequently, changes the robot axes to the solved matrix.

Besides, a function that belongs to the same class was implemented to handle the forward kinematics concept. Whenever this function called, it creates a kinbody and let the user to freely change the position of the kinbody and the robot's joints angles from the simulation window, also it displays the current kinbody's position and the robot's joints angles. The last method that belongs to this class is the behavioral control function. This method just assign consequently to the robot's manipulator multiple tasks to do, for example the robot should pick an object that exists in the environment like a mug and place it in another place. Also this method is called whenever the user choose the required task from a drop-down list in the user-interface.

Python with QT

Qt is a cross platform application framework that is widely used for what is qtdeveloping application software with a graphical user interface (GUI) and non-GUI programs such as command-line tools and consoles for servers. Mainly Qt is written in C++ and is fully object-oriented code. Also it includes a powerful API, rich set of C++ class libraries and tools for GUI development. In addition, it provides an excellent documentation which covers almost everything available in Qt. Qt allows open source and commercial software developers to code less, create more and deploy everywhere. With Qt, developers can build innovative applications and touch-enabled user interfaces once and then deploy across all major mobile, desktop, consumer electronic and embedded platforms without rewriting the code. Qt underpins Nokia's growing developer offering, and its strategy to develop a wide range of products and experiences that people can fall in love with.

PYQt is also an application that belongs to Qt family but has a specific feature. This feature is after writing the object-oriented C++ code for the designed graphical user interface, it can be converted into an object-oriented Python code. And the proposed system, PYQt was used in order to generate a Python code for the graphical user interface, and consequently uses this program to connect and interact with the main Python code and OpenRave server.

Focusing on the proposed Python-Qt program and its main objective which is implementing, designing, and displaying a graphical user interface (GUI), it was created firstly by using Qt-Designer program to implement the design. Then convert the UI output file to Python file, this python code that was generated just implements and displays whatever was designed with Qt-Designer program but with object-oriented python code but Qt libraries must be imported firstly in the code.

There exists a "MainWindow" class in the PYQt program, this class is responsible for initializing a new "Ui-Form" class, the main class which actually draw the graphical user interface window and its contents. The more important objective for this class is connecting each and every object in the user interface with a specific function which implements the specific task for this object. For example, those objects could be buttons, spin-boxes, dials or drop-down lists. In addition, in the graphical user interface, it exists a text-browser item that displays the events and the updates of the simulation and the running program.

Finally, there are two functions in the same class, "SentToServer()" and "SendToOr()", their task is to inform and send signals and commands to the main server and OpenRave server. Those are considered actually the connection between the graphical user interface and the main Python program as well as OpenRave program.

Chapter 5

Experimental Study

5.1 Experimental Setup

Specifically, the experiment has been tested on Ubuntu 11.04 operating system (An operating system based on Debian Linux distribution and packages) for a 32 bit system (The number of bits in which the processor can handle the instructions that exist in the random-access-memory). Furthermore, it has been tested on i-3 Core system and a 2.13 Giga-Hertz processor. Moreover, the capacity of the Random-Access-Memory (RAM) of the system that was testing the software is 4 Giga-Byte.

5.2 Evaluation Metrics

5.2.1 Quantitative Evaluation

For this experiment, two evaluations has been done, quantitative and qualitative evaluation, especially because most researches and evaluators are convinced that whenever a quantitative and qualitative evaluations and analysis are done on a specific project, it produces more comprehensive understanding about that system.

Mainly, the reasons for doing a quantitative evaluation and analysis for the system are measuring and testing the performance and the efficiency of that system, predict real-world events, and display the results in a mathematical form like a line graph, bar graph, pie chart or Venn diagram. Concerning the proposed system, we will cover and provide some evaluation metrics like relative point position, joints angle, environment complexity, success rate or reliability, and time taken by the robot to accomplish a certain process

or a command. Thus, for evaluating the proposed system, we will use two variables separately, and fix the other variables.

Point Position vs Success Rate

As shown in Figure 5.1, a line graph about the relative point position (a moving point relative to the robot's base) with the success rate or the reliability and the robot reachability for that point in the 3-D environment was proposed. This evaluation has been done based on the inverse kinematics function. In detail, the graph shows that firstly the point is close to the base, the reachability of the robot is very low, and then becomes almost 100% and then becomes almost 0 again. Typically this evaluation represents and defines the concept of the reachability region for the robot.

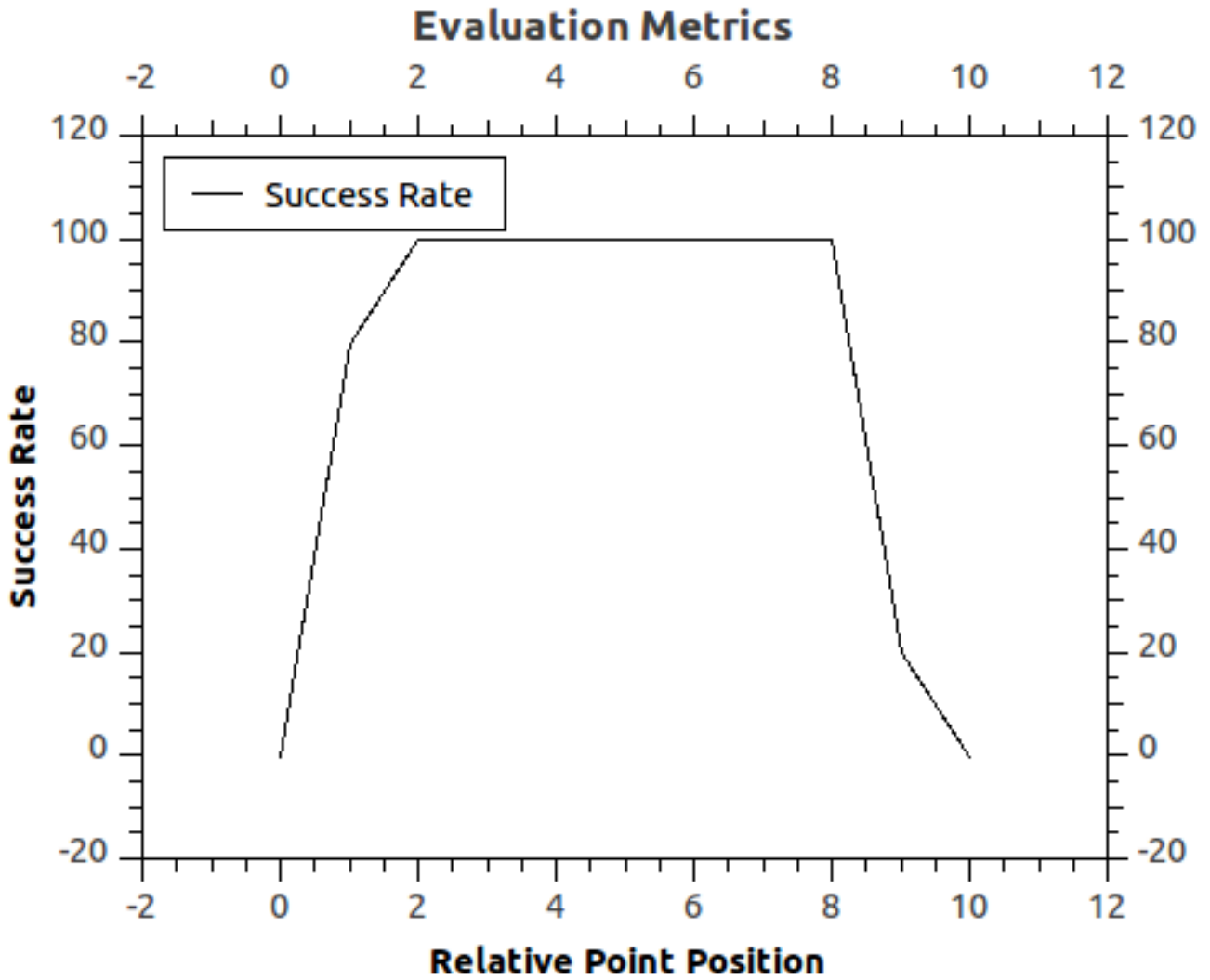


Figure 5.1: Relative Point Position with Success Rate Line Graph

Point Position vs Time

For the inverse kinematics, it is impossible to calculate the time that the robot takes to reach a specific destination and adjust the robot joints angle, since the software calculates the angles and executes the commands to reach a given point separately from the previous points. So the robot does not execute a transformation between a given point and another, therefore it is impossible to calculate the time using inverse kinematics. Besides it is also impossible to calculate and determine the time with the forward kinematics function because, as it is proposed, the process of the forward kinematics is

done instantaneously and there is no delay between moving the kinbody and the robot process time.

Joint Angle vs Success Rate

As shown below in the relative joints angle versus the success rate line graph, there is a vertical line which indicates that joint angles have always succeeded without any failure. However, it is highly important that we should avoid the robot self-collision and the collision between the environment and the robot itself because that's not our case. Also this evaluation has been done using the direct control.

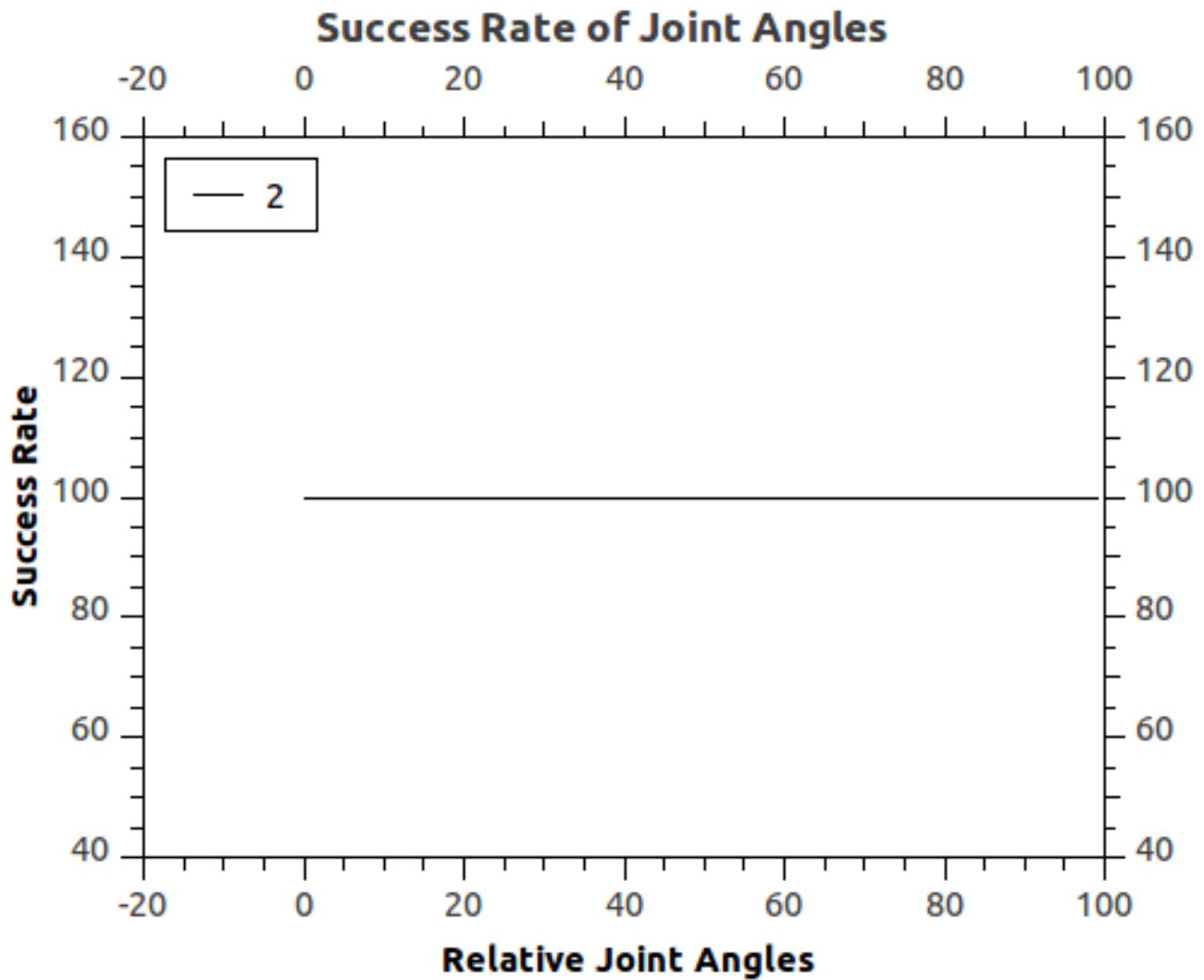


Figure 5.2: Relative Joint Angles with Success Rate Line Graph

Joint Angle vs Time

The relative joint angles with the time line graph describes that whenever the joint angle of the robot increases, the time taken to reach the final position with the new angle increases too.

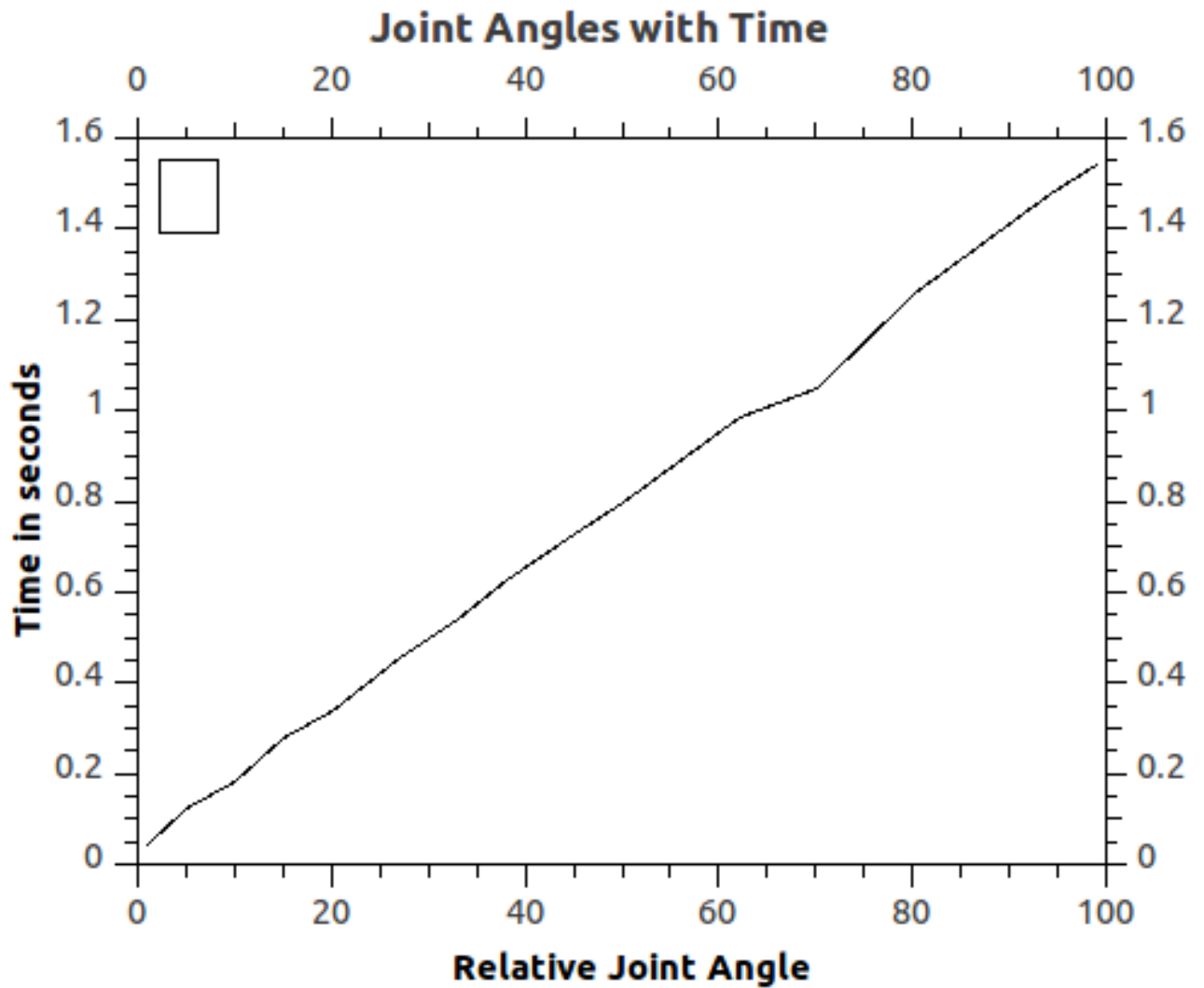


Figure 5.3: Relative Joint Angles with Time Line Graph

Environment Complexity vs Success Rate

Figure 5.4 represents a line graph for the environment complexity with the success rate of executing the process. So it shows that the incremental values of the environment complexity are accompanied by decremental values of the success rate and the system reliability.

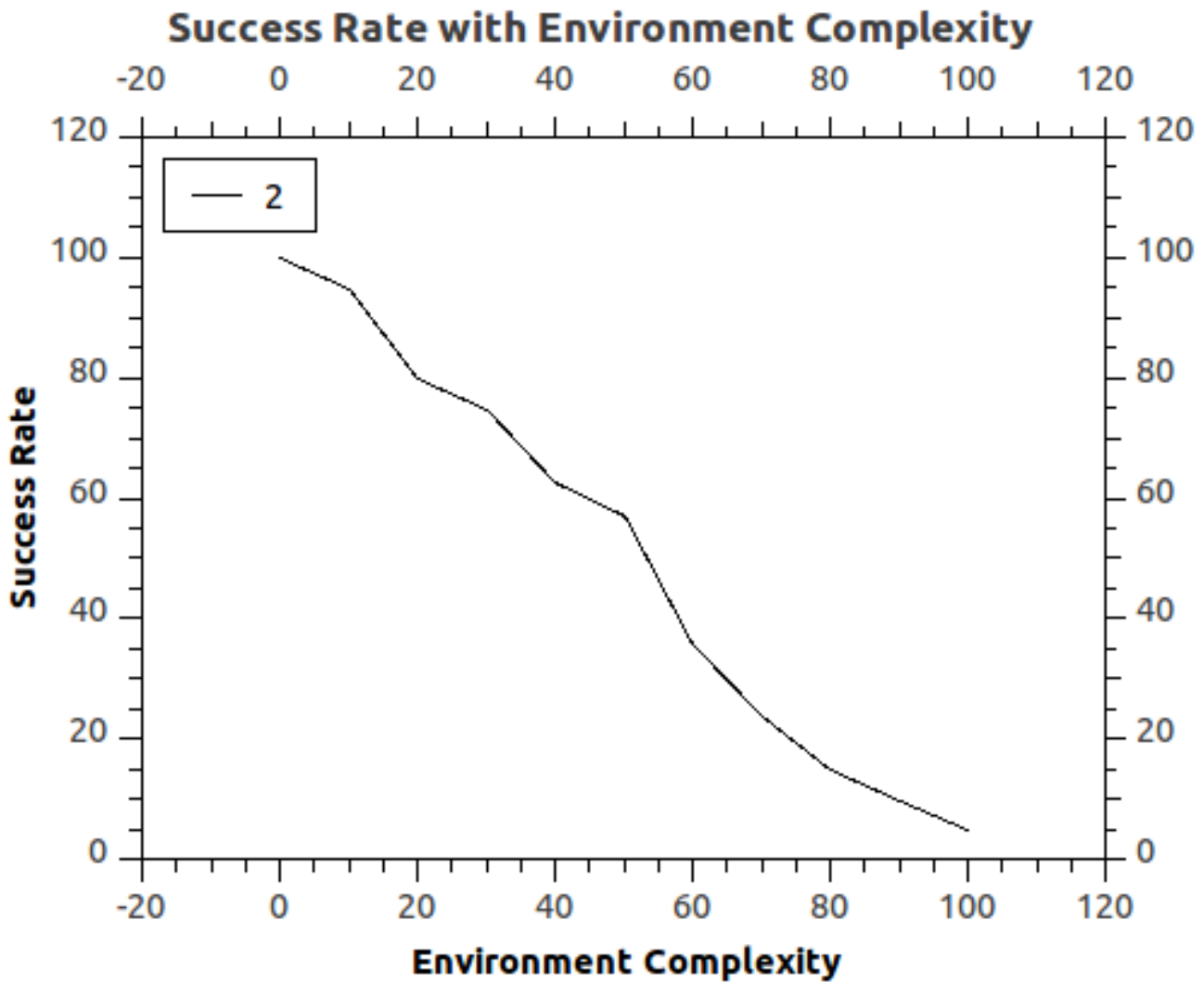


Figure 5.4: Environment Complexity with Success Rate Line Graph

Environment Complexity vs Time

In the represented graph shown in Figure 5.5, it is clear that the more complex the environment is, the more the time that the robot takes to accomplish a certain given process. That's because when the robot exists in a complex environment, it takes more time to determine the path to the given destination without any collision with an object which exists in the environment, so it can take a long path to reach the given destination with no collision. On the other hand, when the robot exists in a simple environment, it takes less time to determine the path, and even the path itself is shorter.

However, we should keep in mind in both situations that there could be a robot self-collision.

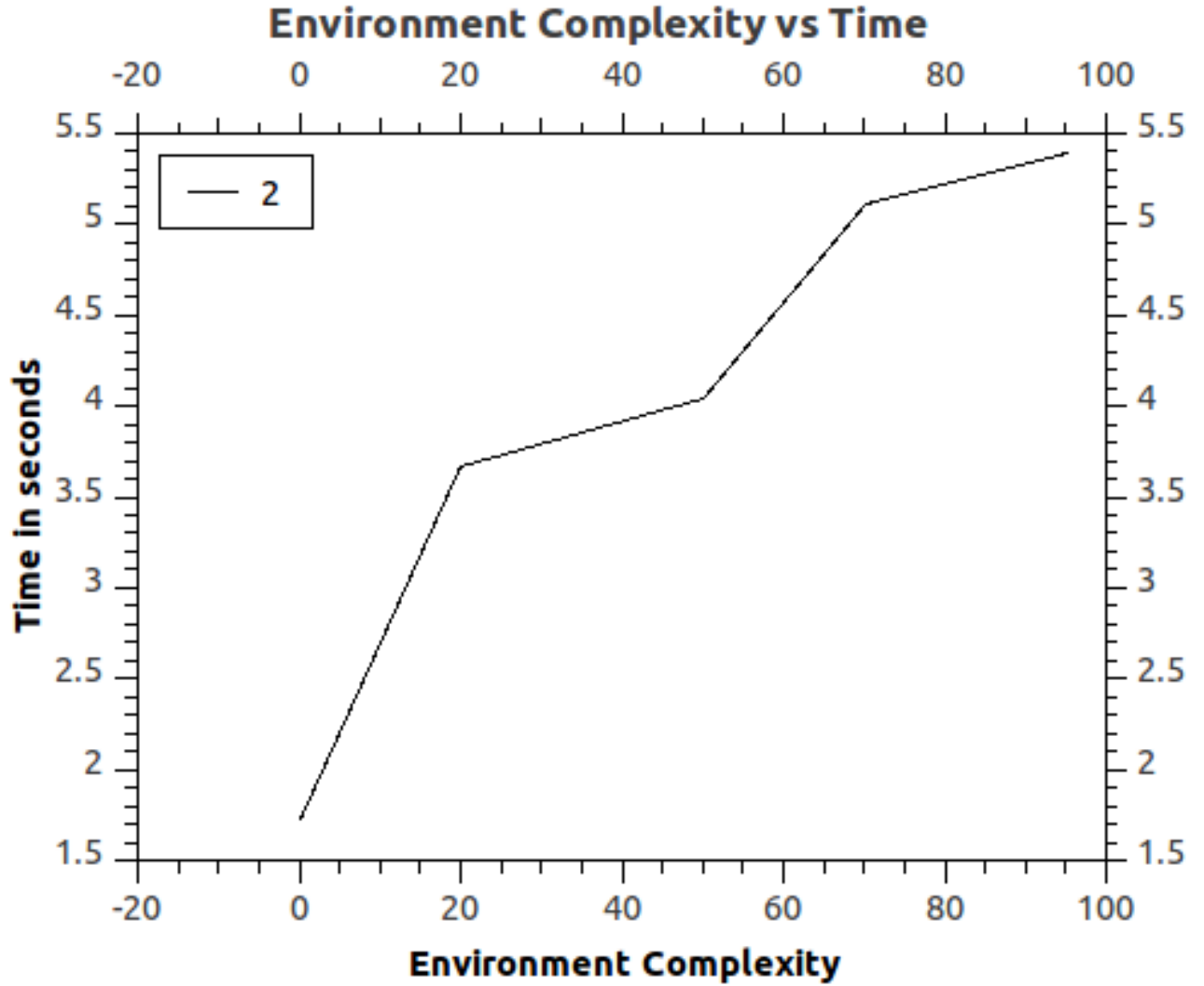


Figure 5.5: Environment Complexity with Time Line Graph

5.2.2 Qualitative Evaluation

Mainly, a qualitative research and analysis is another method or evaluation that can be done on a specific system, but the main reason of doing such evaluation and analysis is to measure and clarify the quality of the proposed software. Moreover, the qualitative analysis determines and shows the attitude and behavior of the system itself. Also, it uses subject judgment

based on non-quantified information. Basically, it differs from the quantitative evaluation, since it focuses more on the value and the behavior not on the numbers.

General Control Panel

The figure below shows the main panel that appears whenever the program runs. Mostly, the main frame contains a tab-widget container, the first tab is the configuration panel tab, it is responsible for opening the OpenRave server and also loading and running the environment. Moreover, there exists a Close button which can close all the systems. Furthermore, the same tab contains a status text browser that displays and informs the user what he has done and what are the commands that he executes.

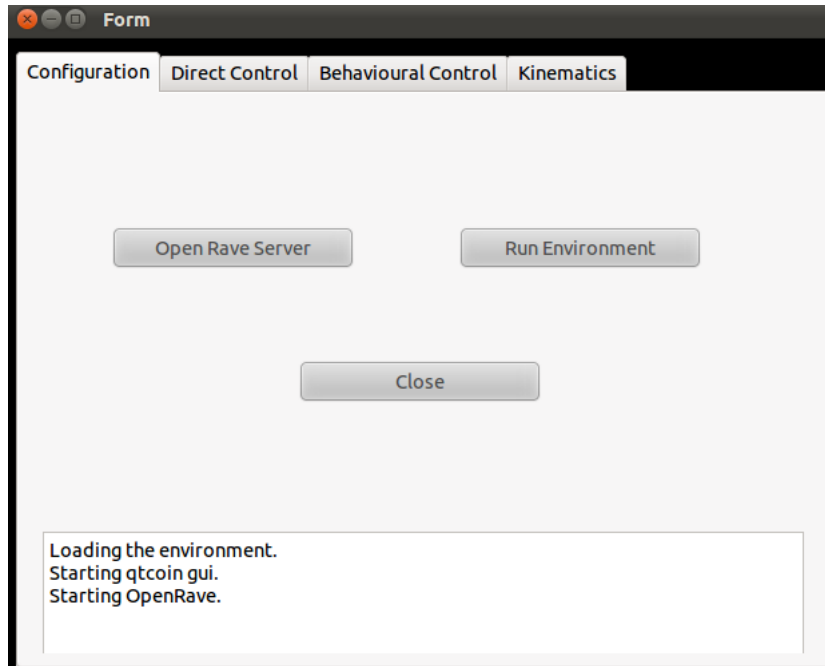


Figure 5.6: General Control Panel

Environment Complexity

The figures below (5.7, 5.8, 5.9 and 5.10) display the OpenRave panel, the environment that was loaded and the 7-DOF Barrett-Wam robot arm. Besides they show that the user can play-around and change the complexity of the environment itself by replacing the objects that surround Barrett-Wam.



Figure 5.7: Barrett-Wam in minimum complexity environment

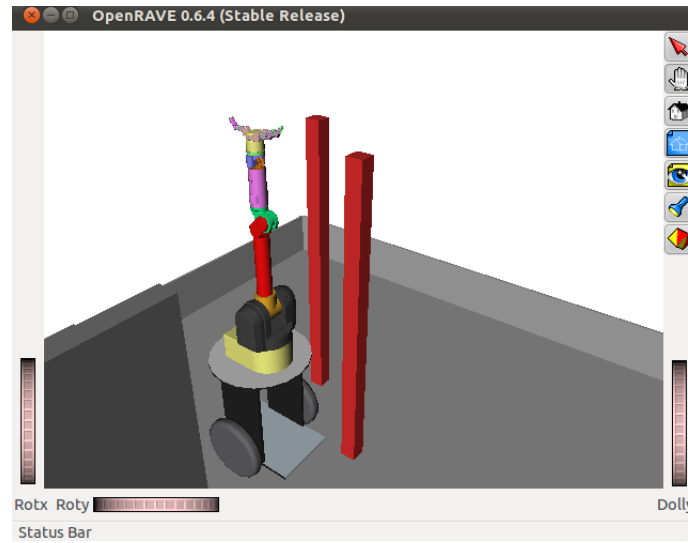


Figure 5.8: Barrett-Wam in medium complexity environment

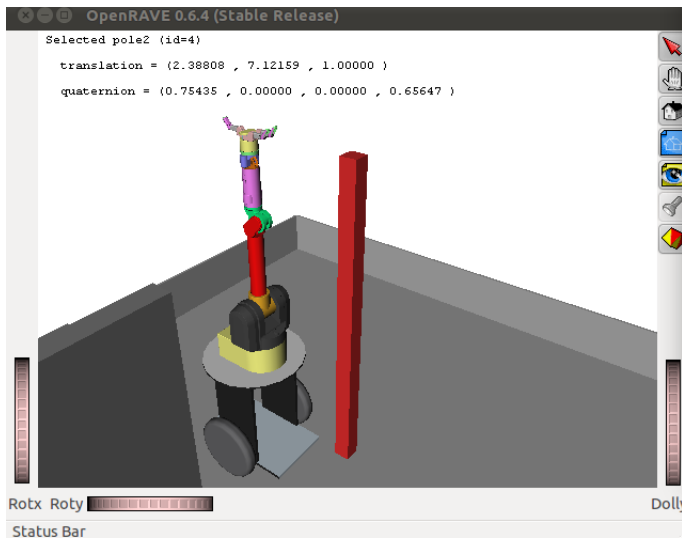


Figure 5.9: Barrett-Wam in medium complexity environment

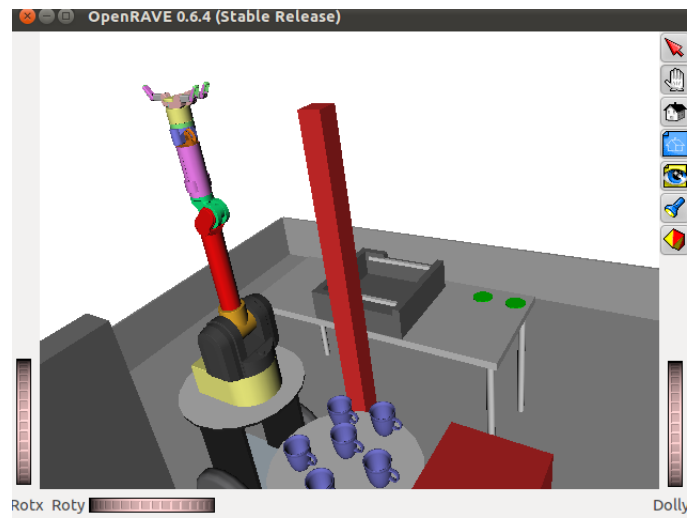


Figure 5.10: Barrett-Wam in maximum complexity environment

Direct Control

Figure 5.11 shows the direct control panel, in this panel exists 6 dials and spin-boxes, each dial with a spin-box are responsible for a specific joint angle in the loaded Barrett-Wam robot, so whenever the user changes the value of

a certain dial or spin-box, the specific joint angle changes instantaneously. Moreover, there exists a combo-box which can change the situation of the robot arm's fingers, opening or closing them. Looking at Figures 5.12, 5.13, 5.14, 5.15 and 5.16, they show the effects of the direct control commands that was given by the user.



Figure 5.11: Direct Control panel

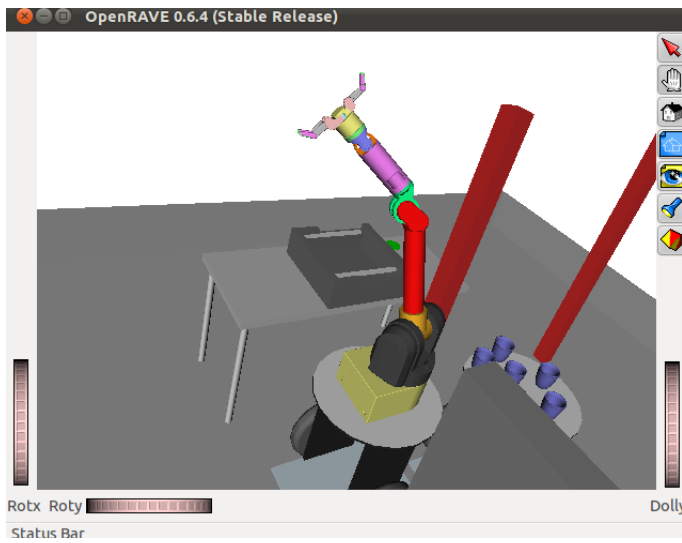


Figure 5.12: Example shows Direct Control - Opened Fingers

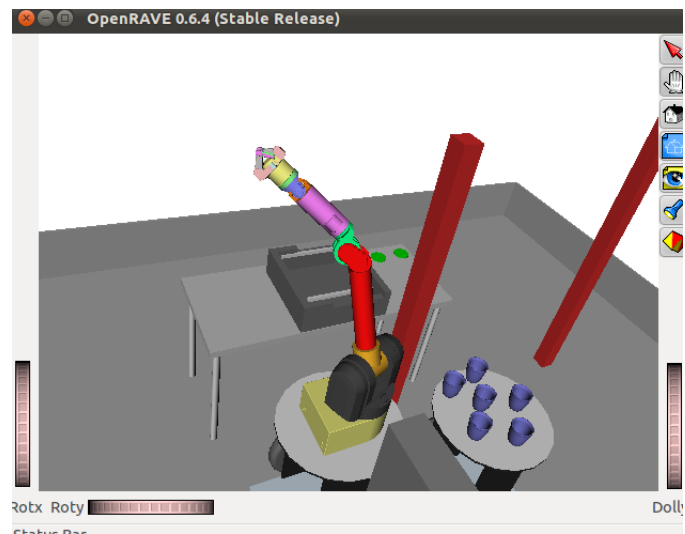


Figure 5.13: Example shows Direct Control - Closed Fingers

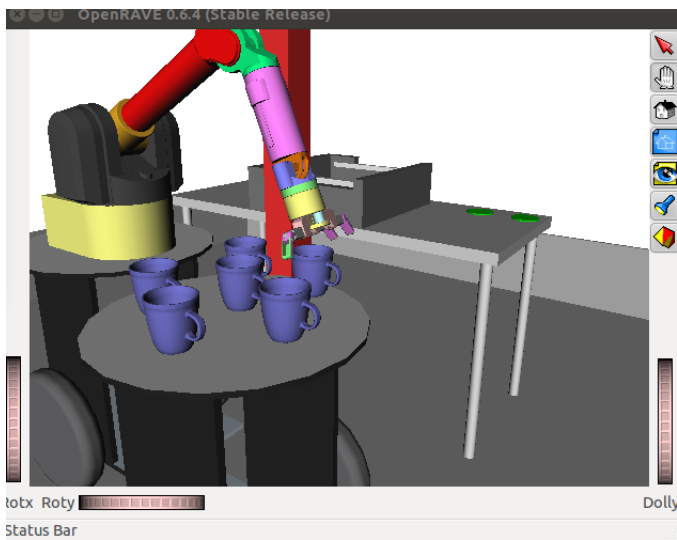


Figure 5.14: Example shows Direct Control-Object Grasping

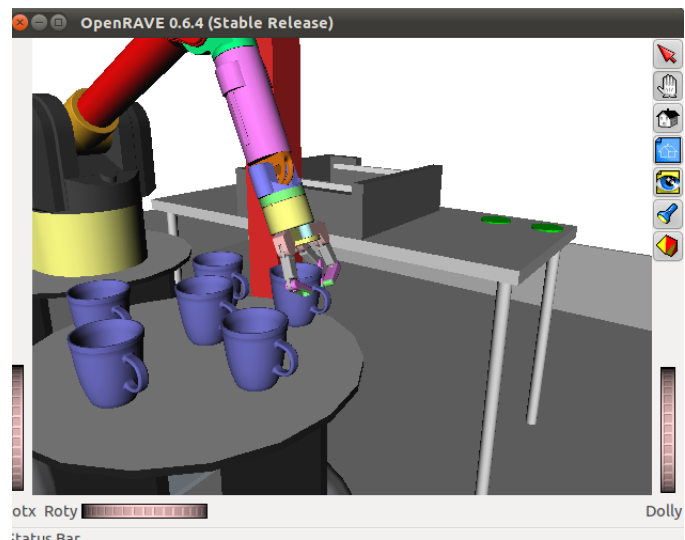


Figure 5.15: Example shows Direct Control-Object Grasping

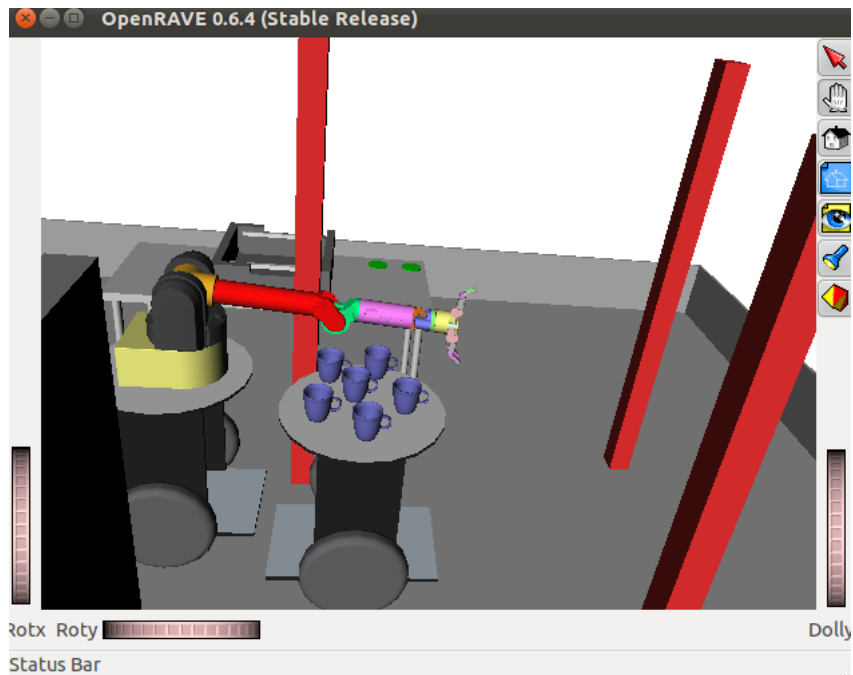


Figure 5.16: Example shows Direct Control with joint 2 with angle equals 82 units

Forward Kinematics

An evaluation for the forward kinematics function has been done and displayed in Figures 5.17, 5.18, 5.19, 5.20 and 5.21. So it consists of a free control, which allows the user to freely move a 3-D object and displays the robot joints angle and the current 3-D location of the object.

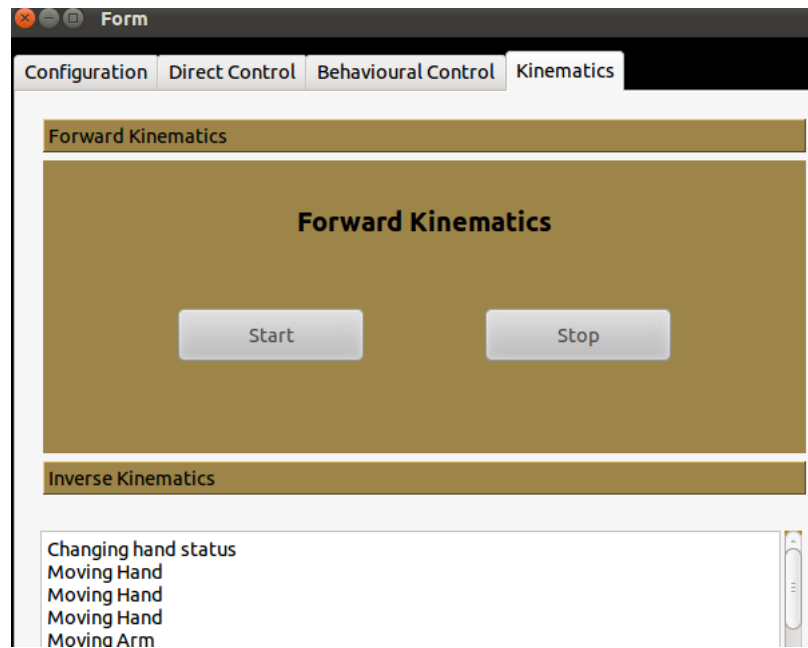


Figure 5.17: Forward Kinematics panel

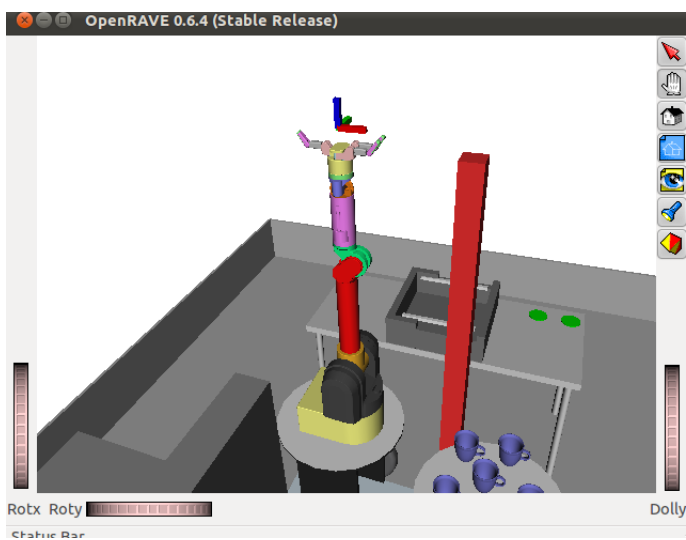


Figure 5.18: Forward Kinematics Example 1

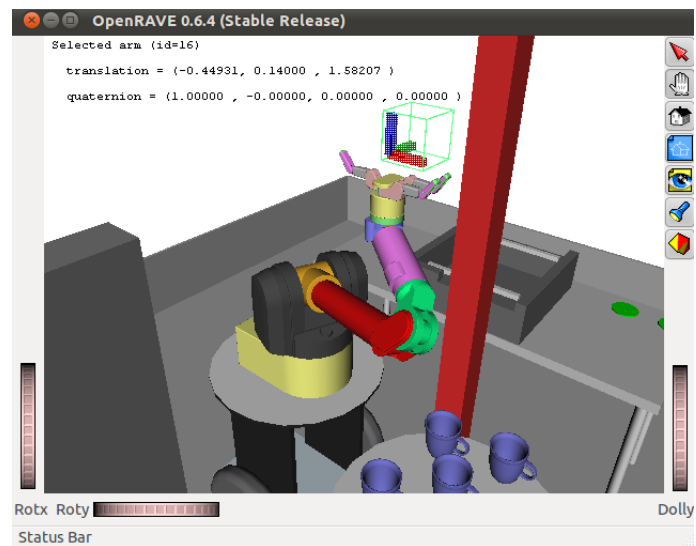


Figure 5.19: Forward Kinematics Example 2

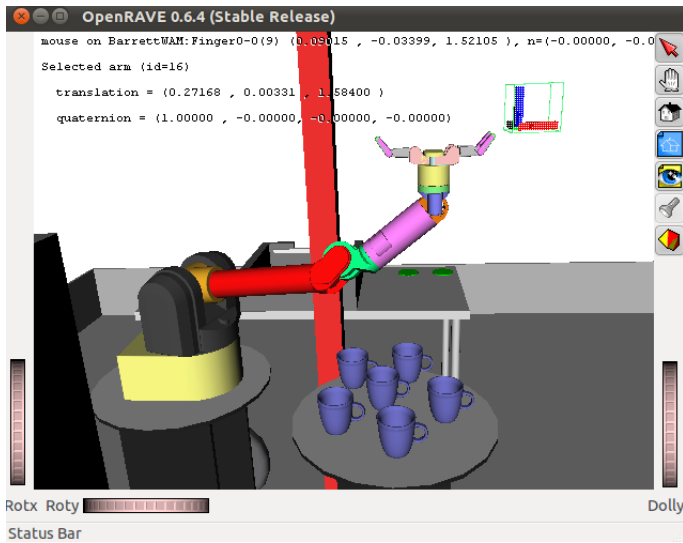


Figure 5.20: Forward Kinematics - Unreachable destination Example

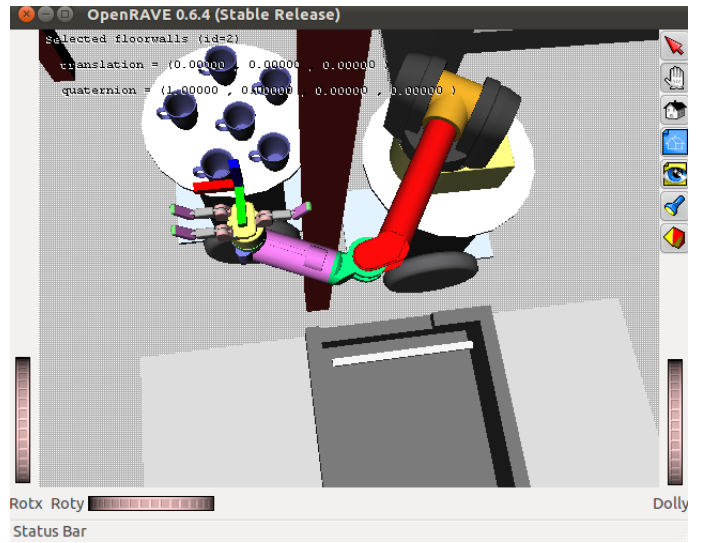


Figure 5.21: Forward Kinematics Example - Top View Camera

Inverse Kinematics

Figures 5.22 and 5.23 show the inverse kinematics evaluation that has been done to the system, so it displays the inverse kinematics panel and OpenRAVE example respectively. In the inverse kinematics panel, there exists 3 spin-boxes, that are responsible for x, y, and z axis respectively. It just indicates the object coordinates. The user can change these coordinates which are relative to the robot's base location, and then apply the inverse kinematics formula on this point. If the point is in the reachability region of the robot arm, so it adjusts the robot joints angle to reach this point, if not, it alerts unreachable destination message to the user.

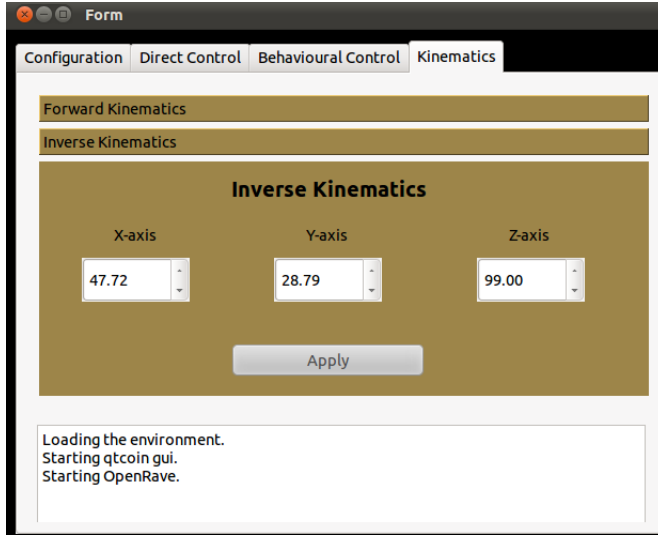


Figure 5.22: Inverse Kinematics Panel

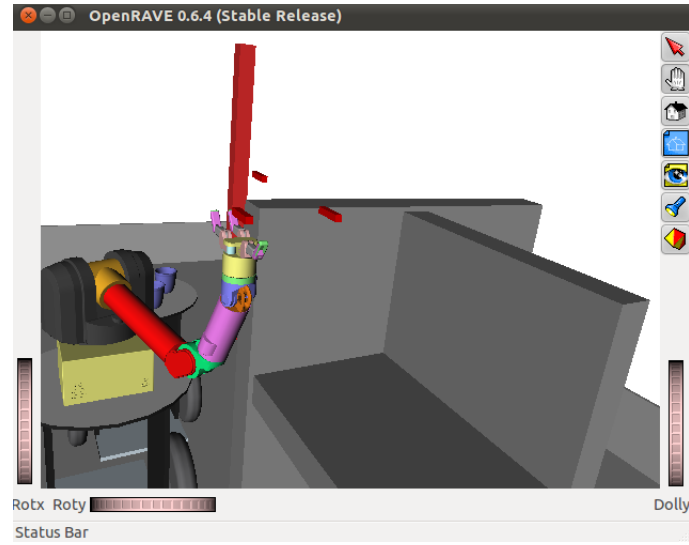


Figure 5.23: Inverse Kinematics Example

5.3 Results and Discussion

The experiment above represented the quantitative and qualitative evaluations. In the quantitative analysis, it is showed that the time of the process differs with the location of the joint in the 3-D space, it also differs with the robot's joints angle and the complexity of the environment and the surrounding objects. Moreover, it has represented the success rate, in another word the reliability of the proposed system.

Furthermore, the qualitative analysis declared the behavior of the system and showed that the system is running perfectly and easy for any user to deal with navigate through the 3-D environment and the control panel.

Chapter 6

Conclusion and Future Work

6.1 Conclusion

In conclusion, the simulation of a industrial robot in a specific environment become nowadays a fundamental and primary issue for industrial robotics field, and not just a secondary issue. And that's because it helps the users to test firstly the robot arm behavior in the virtual environment before applying that in the real world, so that mainly helps in error avoidance.

The simulation could also help the users to give the virtual and the real robots some specific tasks through the program, and then both of them executes them these tasks accurately without any fault. Therefore, it lets the users to give the robot some orders with contacting only the program on the computer, and do not have to physically contact the robot arm itself or even exist in robot environment, as it shown in the proposed system.

6.2 Future Work

the system could be developed in order to achieve a more professional program in the 3-D robotics simulation field. For example, the system could have the ability of letting the user choose from a list the robot arm which will be loaded and interacting with the environment, not just the Barrett-Wam robot arm.

Besides, the program could be developed and let the user select the number of the robot's DOF, in other words, lets the user control which joints in the robot arm will be activated and which joints will be deactivated.

Furthermore, the behavioral control in the system could be developed, it may handle other complex processes which could fit more in industrial environments. The program could be integrated also to contain more than one robot arm, and the software should make sure that the robots should cooperate together.

As it's mentioned before, any 3-D simulation system could be integrated on a real robot arm. So in the future work for the proposed system, it could connected to a real robot arm and let the user control it.